A Quantum Natural Language Processing Approach to Musical Intelligence

13

Eduardo Reck Miranda, Richie Yeung, Anna Pearson, Konstantinos Meichanetzidis, and Bob Coecke

Abstract

There has been tremendous progress in Artificial Intelligence (AI) for music, in particular for musical composition and access to large databases for commercialisation through the Internet. We are interested in further advancing this field, focusing on composition. In contrast to current 'black-box' AI methods, we are championing an *interpretable compositional* outlook on generative music systems. In particular, we are importing methods from the Distributional Compositional Categorical (DisCoCat) modelling framework for Natural Language Processing (NLP), motivated by musical grammars. Quantum computing is a nascent technology, which is very likely to impact the music industry in time to come. Thus, we are championing a Quantum Natural Language Processing (QNLP) approach to develop a new generation of intelligent musical systems. This work follows previous experimental implementations of DisCoCat linguistic models on quantum hardware. In this chapter, we present *Quanthoven*, the first

E. R. Miranda (✉)
ICCMR, University of Plymouth, Plymouth, UK
e-mail: eduardo.miranda@plymouth.ac.uk

E. R. Miranda · R. Yeung · A. Pearson · K. Meichanetzidis · B. Coecke
Quantinuum, Oxford, UK

R. Yeung
e-mail: richie.yeung@cambridgequantum.com

Anna Pearson
e-mail: anna.pearson@cambridgequantum.com

Konstantinos Meichanetzidis
e-mail: k.mei@cambridgequantum.com

Bob Coecke
e-mail: bob.coeck@cambridgequantum.come

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

E. R Miranda (ed.), *Quantum Computer Music*,
https://doi.org/10.1007/978-3-031-13909-3_13

proof-of-concept ever built, which (a) demonstrates that it is possible to program a quantum computer to learn to classify music that conveys different meanings and (b) illustrates how such a capability might be leveraged to develop a system to compose meaningful pieces of music. After a discussion about our current understanding of music as a communication medium and its relationship to natural language, the chapter focuses on the techniques developed to (a) encode musical compositions as quantum circuits, and (b) design a quantum classifier. The chapter ends with demonstrations of compositions created with the system.

13.1 Introduction

When people say that John Coltrane's *Alabama* is awesome or that Flow Composer's[1] *Daddy's car* is good, what do they mean by 'awesome music' or 'good music'? This is debatable. People have different tastes and opinions. And this is true whether the music is made by a human or a machine.

The caveat here is not so much to do with the terms 'awesome music' or 'good music', or whether it is made by humans or machines. The caveat is with the word 'mean'.

In the last 20 years or so, there has been tremendous progress in Artificial Intelligence (AI) for music. But computers still cannot satisfactorily handle meaning in music in controlled ways that generalise between contexts. There is AI technology today to set up computers to compose a decent pastiche of, say, a Beethoven minuet; e.g., there are connectionist (a.k.a., 'neural networks') systems for composition that have been trained on certain genres or styles. (For a comprehensive account of the state of the art of AI for music, please refer to [46].) However, it is very hard to program a computer to compose a piece of music from a request to, say, 'generate a piece for Alice's tea party'. How would it know what tea party music should sound like, or who Alice is? And how would it relate such concepts with algorithms to compose?

A challenging task is to design generative systems with enough knowledge of musical structure, and the ability to manipulate said structure, so that given requests for mood, purpose, style, and so on, are appropriately met. Systems that currently attempt to perform such tasks, especially for music recommendation systems, work in terms of finding and exploiting correlations in large amounts of human-annotated data; e.g., [2]. These for example would fill a preference matrix, encoding information such as 'likes' for songs, which are categorised already in certain genres by listeners.

Following an alternative route, which comes under the umbrella of *Compositional Intelligence* [15], we are taking the first steps in addressing the aforementioned

[1] Flow Composer is an AI lead sheet generator developed at Sony CSL Paris [53]. A lead sheet is a form of musical notation that specifies the essential elements (melody, lyrics, and harmony) of a song.

challenge from a Natural Language Processing (NLP) perspective, which adopts a structure-informed approach.

Providing domain-specific structure to intelligent systems in a controlled and scalable way is a nontrivial challenge: we would need to ensure that the system does not significantly lose the flexibility to adapt to different datasets. Nevertheless, having structure present in intelligent systems has potential benefits, such as increased interpretability. Another potential benefit is the need for fewer training parameters; in general, a learning system would need a set of free parameters so that it learns structure from scratch. But in our case, a structural approach is motivated by an analogy between grammar in language, and structure in musical compositions. Specifically to the present work, we are pioneering a Quantum Natural Language Processing (QNLP) approach to develop a new generation of intelligent musical systems. This in turn is motivated by our specific choice of mathematical grammar and a modelling framework for NLP, referred to as Distributional Compositional (DisCo). DisCo models are amenable to implementation on quantum computers due to a shared mathematical structure between grammar and quantum theory.

Our aim in this work is to train parameterised quantum circuits to learn to classify music conveying different meanings. And then, use the classifier to compose music that conveys such meanings via rejection sampling. Here, the notion of 'meaning' is defined as *perceived properties of musical compositions, which might holistically characterise different types of music*. This is the first step towards the ambition of being able to ask an autonomous intelligent system to generate bespoke pieces of music with particular meanings, required purposes, and so on.

Our classification system is trained on human-annotated examples. It scans given examples for musical elements, which we refer to as *musical snippets*. A snippet can be as small as a single musical note. But it can also be a melody, a bar of music, or a group of bars. The main assumption made by the system—known also as the *distributional hypothesis*—is that snippets that occur in similar contexts convey similar meanings. This is a hypothesis that regards the meanings of words in a text: words that appear in similar contexts convey similar meanings. Then, a combination of the meanings of all snippets determines the meaning of entire compositions, or sections thereof.[2] These combinations follow *grammatical* rules. Our aim is to input these rules into the machine along with meaning-encodings of the snippets, to allow it to identify different types of music. In the near future, we hope it will be able to use this knowledge to compose pieces, or sections thereof, conveying specific meanings. Towards the end of the chapter, we provide a glimpse of how this might be achieved.

Technically, the system assigns types to snippets, in analogy with a parser assigning grammatical part-of-speech tags to words, such as nouns, adjectives, verbs, and so on. Types follow specific compositional rules, and in the specific grammar model that we employ—that is, *pregroup grammar*—types obey algebraic relations.

[2] For large pieces of music, it is assumed that different sections may convey different meanings, in the same way, that sentences or paragraphs convey localised meanings within the overall narrative of a large text.

The rules induce a dependency structure onto a musical piece that is composed with snippets. Then, *meaning* enters the picture by sending types to vector spaces of specific dimensions. Snippet-meaning is encoded inside these vector spaces in terms of multilinear maps, i.e., *tensors*. The compositional rules represented by the dependency structure induce a pattern of tensor-contractions between the tensors.

The choice to embed meaning in vector spaces is motivated by a similar mathematical structure between the algebra of the types and the composition of tensors. Importantly, the tensors form a network, which reflects the dependencies as *inherited* by the grammatical structure of a piece of music. However, as we are encoding meaning in tensors, we realise that the computational cost of manipulating and composing tensors is prohibitive for large numbers of snippets, even for mainframe supercomputers, let alone your average desktop. These would require high dimensionality of vector spaces along with a highly connected dependency network.

Enter the quantum computer! Quantum processes constitute a powerful choice for a playground in which to encode meaning. After all, quantum theory can be formulated entirely in terms of complex-valued tensor networks. Specifically, quantum theory can be fully considered around one fundamental principle: the way quantum systems compose to form larger ones, and the mathematical operation that jointly describes multiple quantum systems is the tensor product.

Emerging quantum computing technology promises formidable processing power for some tasks. As it turns out, and we will see this below, the mathematical tools that are being developed in QNLP research are quantum-native, by yet another analogy between mathematical structures. That is, they enable direct correspondences between language and quantum mechanics, and consequently music. In this case, (a) the meaning of words can be encoded as quantum states and (b) grammars would then correspond to quantum circuits [12].

Despite the rapid growth in size and quality of quantum hardware in the last decade, quantum processors are still limited in terms of operating speed and noise tolerance. However, they are adequate to implement proof-of-concept experiments of our envisaged system.

In this chapter, we present *Quanthoven*, the first proof-of-concept ever built, which (a) demonstrates that it is possible to program a quantum computer to learn to classify music that conveys different meanings and (b) illustrates how such a capability might be leveraged to develop Artificial Intelligence systems able to compose meaningful pieces of music.

The remainder of this chapter is structured as follows:

- Section 13.2 explores our current understanding of music as a communication medium and its relationship to natural language, which motivates our research.
- Section 13.3 contextualises our work in relation to approaches to computational modelling of music and algorithmic composition.
- Section 13.4 introduces the very basics of quantum computing, just enough to follow the technical aspects of the quantum classifier (i.e., machine learning algorithm) discussed in this chapter.

- Section 13.5 introduces Distributional Compositional Categorical (DisCoCat) modelling, which uses Category Theory to unify natural language and quantum mechanics. It also shows how DisCoCat can model music.
- Section 13.6 describes the DisCoCat music model that we developed for this project, including the process by which musical pieces are generated using a bespoke grammar and converted into a quantum circuit. It delineates the machine learning algorithm and training methodology.
- Section 13.7 illustrates how the results from the classification can be leveraged to generate two classes of musical compositions.

The chapter ends with final remarks. A number of appendices provide complementary information.

13.2 Music and Meaning

It is widely accepted that instrumental music[3] is a non-verbal language. But what does music communicate? What is meaning in music?

Some advocate that music communicates nothing meaningful because it cannot express ideas or tell stories in the same way that verbal languages can. Music has no 'words' to express things like 'Bob', 'mussels and fries', 'beer', 'Bob loves mussels and fries with a beer', and so on. But this is a rather defeatist view.

Conversely, others advocate that music can communicate messages of some sort. For example, the imitation of cuckoo birds in Ludwig van Beethoven's Pastoral Symphony is often thought to signify 'springtime has arrived' [49]. But this is speculation. We do not know if Beethoven really wanted to communicate this.

There have been studies, however, suggesting that different types of chords, rhythms, or melodic shapes may convey specific emotions. For instance, the notion that a minor chord conveys sadness and a major one happiness has been demonstrated experimentally [4]. And brain imaging studies have identified correlations between perceived musical signs and specific patterns of brain activity associated with feelings. These studies support the notion that music conveys, if anything, affective states [18,33]. But still, there is no treatise to date on how to communicate a specific feeling in a composition. There are no universal rules for doing this.

Yet, it would be perfectly possible to have musical languages matching the expressive power of verbal languages. Creators of Esperanto-like verbal languages, such as Dothraki, made for the TV series *The Game of Thrones*, have developed fully-fledged musical languages [50,57]. It is just that *Homo sapiens* have not naturally evolved one as such. This is because we evolved musicality for purposes other than verbal communication. Thus, meaning in music is not quite the same as meaning in

[3] That is, music without singing. No lyrics. Only musical instruments.

verbal languages. But what is it? To tackle this question, we ought to examine how the brain makes sense of language and music.

13.2.1 Brain Resources Overlap

Music and verbal languages do have a lot in common. There is scientific evidence that the brain resources that humans deploy to process music and verbal languages overlap to a great degree [29]. Stefan Koelsch proposed an ambitious neurocognitive model of music perception supported by extensive research into the neural substrates shared by music and language [32]. Of course, there are differences too. For instance, whereas, the brain engages specialised areas for processing language (e.g., Broca's and Wernicke's areas), music processing tends to be distributed over a number of areas, which are not necessarily specialised for music.

Let us consider, for instance, the notion of narrative, in a text and in a piece of music; see also discussion in Chap. 2 in this volume. We seem to deploy cognitive strategies to make sense of music, which are similar to those employed to read a text. This is because verbal languages and music share similar intrinsic structures [54]. Or at least our brain thinks that this is the case. This is questionable. But it is a useful assumption, which is somewhat supported by experiments testing if musical cognitive functions can influence linguistic ones, and vice-versa.

Notable experiments highlighted the benefits of musical proficiency for the acquisition of linguistic skills, in particular learning a second language [40,55]. And Dawson et al. provided experimental evidence that native speakers of distinct languages process music differently [19]. This seems to be determined by the structural and phonological properties of their respective languages. For example, speakers of languages that has words whose meaning is determined by the duration of their vocalization tend to deal with rhythm more accurately than speakers of languages whose durations do not interfere with meaning.

13.2.2 Meaning is Context

In a similar fashion to a narrative in text, a musical narrative delineates relationships and changes in the state of affairs from event to event [30]. Auditory events in a piece of music acquire significance within a structure that our mind's ear imposes on them. Different types of music define systems of relations among these elements, which induce us to assign them structural impressions; or create categories. The more we listen to the music of certain styles, the more familiar the categories that define such styles become. And the higher the chances that those assigned impressions might be elicited when we listen to such kinds of pieces of music again and again.

Thus, we parse and interpret auditory streams according to our own made-up mental schemes. We might as well refer to such schemes as *musical grammars*. And the labels we give to impressions as *meanings*.

A piece of music may as well make one feel sad or happy, or remember Bob, or think of mussels and fries. And one might even hear Beethoven shouting from his grave that springtime has arrived. Thus, for instance, music with fast repetitive patterns might be categorised as 'rhythmic', 'energetic', or 'exciting'. In contrast, music composed of irregular successions of notes forming melodies and exquisite harmonies might be categorised as 'melodic', 'relaxing' or 'impressionistic'. But this is not as simple as it appears to be. What happens when a piece is prominently rhythmic but is also melodic; e.g., ballroom dance music? We could think of a new category. But what if yet another piece is only slightly more melodic than rhythmic? The boundaries of such categories are not trivial to establish.

Anyway, one thing is certain: the more culturally related a group of listeners are, the higher the chances that they might develop shared repertoires of mental schemes and impressions.

Incidentally, we learn verbal languages in a similar way to what we just described. We make up the meaning of words and learn how to assemble them in phrases from the context in which they are used. We do this as we grow up and continue doing so throughout our lifetime [21]. Perhaps the main difference is that meaning in verbal languages needs to be more precise than meaning in music. That is, meaning in music tends to be more fluid than in language. As stated above, however, one can construct musical natural languages if one attempts to.

13.3 Computational Modelling and Algorithmic Composition

Scholars from both camps (i.e., linguistics and musicology) have developed comparable models to study linguistic and musical structures, and respective mental schemes [5,10,24,35].

Not surprisingly, computational models of linguistic processes have been successfully used to model musical processes and vice-versa. David Cope harnessed transition networks grammars [16], which have been used for the analysis and synthesis of natural language [64], to develop systems to analyse and synthesise music. A system to generate music using stochastic context-free grammars was proposed by [56]. Indeed, Noam Chomsky's context-free grammar [27] proved to be relevant for music in many ways. In particular transformative grammars [9], which suit well the widely adopted method of composing music referred to as variations on a theme. And [42] created a system whereby a group of interactive autonomous software agents evolved a common repertoire of intonations (or prosodies) to verbalise words. The agents evolved this by singing to each other. Moreover, evolutionary models informed by neo-Darwinism [25] have been developed to study the origins of language and music almost interchangeably [7,48].

Generally, there have been two approaches to designing computer systems to generate music, which we refer to as the Artificial Intelligence (or AI) and the algorithmic approaches, respectively [41,44].

The AI approach is concerned with embedding the system with musical knowledge to guide the generative process. For instance, computers have been programmed with rules of common practice for counterpoint and voicing in order to generate polyphonic music. And machine-learning technology has enabled computers to learn musical rules automatically from given scores, which are subsequently used to generate music [17]. The linguistic-informed modelling mentioned above falls in this category.

Conversely, the algorithmic approach is concerned with translating data generated from seemingly non-musical phenomena onto music. Examples of this approach abound. Computers have been programmed to generate music with fractals [28] and chaotic systems [26]. And also with data from physical phenomena, such as particle collision [8], and DNA sequences [45].

Aesthetically, the algorithmic approach tends to generate highly novel and rather unusual music; some may not even consider the outputs as music. The AI approach tends to generate imitations of existing music; that is, imitations in the style of the music that was used to teach the AI system. Neither approach can, however, satisfactorily respond to sophisticated requests such as 'generate something for Alice's tea party' or 'compose a tune to make Bob feel energetic'. The QNLP approach that we are developing is aimed at improving this scenario.

13.4 Brief Introduction to Quantum Computing

A detailed introduction to quantum computing is beyond the scope of this chapter. This can be found in [12,51,62]; see also Chap. 1 in this volume. Nevertheless, in this section, we briefly introduce a few basic concepts deemed necessary to follow the discussions in the following sections.

Our starting point is a quantum bit, known as a *qubit*, which is the basic unit of information carrier in a quantum computer. Physically, it is associated with a property of a physical system; e.g., the spin of an electron up or down along some axis. A qubit has a state $|\psi\rangle$, which lives in a 2-dimensional complex vector space, referred to as Hilbert space. The orthonormal basis vectors $|0\rangle$ and $|1\rangle$, related to measurement outcomes 0 and 1, respectively, allow us to write the most general state of a qubit as a linear combination of the basis vectors known as *superposition*: $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ with $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$.

An important aspect of quantum computers is that they are fundamentally probabilistic. This leads to the situation where even if one knows that a qubit is in a state $|\psi\rangle$, one can only obtain measurement outcomes $i = \{0, 1\}$ with probability given by the Born rule $P(i) = |\,\langle i|\psi\rangle\,|^2$, which gives the square of the norm of the so-called amplitude $\langle i|\psi\rangle \in \mathbb{C}$. So, $\langle i|$ is a quantum effect, also known as a 'bra' in Dirac notation, and is the dual vector of the state, or 'ket', $|i\rangle$. For the general single-qubit superposition in the paragraph above, $P(0) = |\alpha|^2$ and $P(1) = |\beta|^2$.

In order to picture a single qubit, imagine a transparent sphere with opposite poles. From its centre, a vector whose length is equal to the radius of the sphere can point

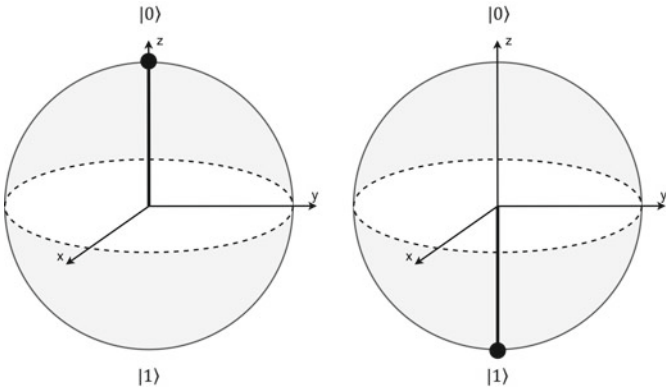


Fig. 13.1 The Pauli **X** gate rotates the state vector (pointing upwards on the figure on the left side) by 180 degrees around the x-axis (pointing downwards on the figure on the right)

to anywhere on the surface. This sphere is called the *Bloch sphere* and the vector is referred to as a *state vector* (Fig. 13.1). Before measurement, the evolution of a single qubit (that is, a qubit that is not interacting with any other qubits) is described by the transformation of its state vector with a unitary linear map U, so that $|\psi'\rangle = U\,|\psi\rangle$ (Fig. 13.2). However if a qubit interacts with other qubits, then things become a little more complicated; more about this below.

In simple terms, quantum computers are programmed by applying sequences of unitary linear maps to qubits. Programming languages for quantum computing provide a number of such linear maps, referred to as *gates*, which act on qubits. For instance, the 'not gate', rotates the state vector by 180° around the x-axis of the Bloch sphere (Fig. 13.1); that is, if the qubit vector is pointing to $|0\rangle$, then this gate flips it to $|1\rangle$, or vice-versa. This gate is often referred to as the 'Pauli **X** gate'. A more generic rotation $\mathbf{Rx}(\theta)$ gate is typically available for quantum programming, where the angle for the rotation around the x-axis is specified. Obviously, $\mathbf{Rx}(\pi)$ applied to $|0\rangle$ or $|1\rangle$ is equivalent to applying **X** to $|0\rangle$ or $|1\rangle$. Similarly, there are $\mathbf{Rz}(\varphi)$ and $\mathbf{Ry}(\theta)$ gates for rotations on the z-axis and y-axis of the Bloch sphere,

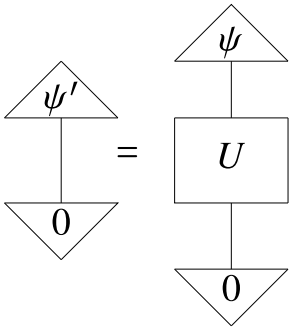


Fig. 13.2 Diagrammatic format showing the evolution of an isolated qubit in initial state $|\psi\rangle$ with unitary map U composed with effect $\langle 0|$. Taken from [36] and used with permission

respectively. An even more generic gate is typically available, which is a unitary rotation gate, with 3 Euler angles: $\mathbf{U}(\theta, \varphi, \lambda)$. Essentially, all single-qubit quantum gates perform rotations, which change the amplitude distribution of the system. And in fact, any qubit rotation can be specified in terms of $\mathbf{U}(\theta, \varphi, \lambda)$; for instance $\mathbf{Rx}(\theta) = \mathbf{U}(\theta, -\frac{\pi}{2}, \frac{\pi}{2})$.

Quantum computation gets really interesting with gates that operate on multiple qubits, such as the controlled **X** gate, or **CX** gate; commonly referred to as the **CNOT** gate. The **CNOT** gate puts two qubits in *entanglement*.

Entanglement establishes a curious correlation between qubits. When considering its action on the computational states, the **CNOT** gate applies an **X** gate on a qubit only if the state of another qubit is $|1\rangle$. Thus, the **CNOT** gate establishes a dependency of the state of one qubit with the value of another. In practice, any quantum gate can be made conditional and entanglement can take place between more than two qubits.

An important gate for quantum computing is the Hadamard gate (referred to as the '**H** gate'). It puts the qubit into a balanced superposition state consisting of an equal-weighted combination of two opposing states: $|\alpha|^2 = 0.5$ and $|\beta|^2 = 0.5$.

A combination of **H** and **CNOT** gates enables the implementation of the so-called Bell states; a form of maximally entangled qubits, which is explored later on in this chapter to represent grammatical structures.

A quantum program is often depicted as a circuit diagram of quantum gates, showing sequences of gate operations on the qubits (Fig. 13.3). So, if one has n qubits, then their joint state space is given by the tensor product of their individual state spaces and has dimension 2^n. The evolution of many qubits interacting with each other is given by a (global) exponentially large unitary map acting on the entire joint exponentially large state space.

There are two ways of thinking of a quantum circuit. On an abstract level, it can be viewed as the application of a linear map to a vector, which computes the entire state of the system at a later time. As stated above, these complex-valued matrices and vectors are exponentially large in terms of the number of qubits required to encode them. Therefore, at least naive simulations of quantum systems are believed to be a

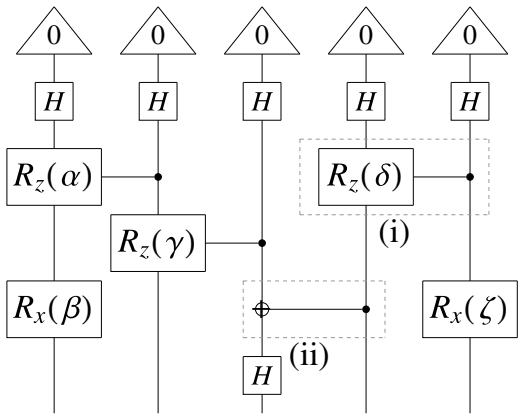


Fig. 13.3 Example of a quantum circuit of the kind used in our system: the Hadamard gate **H**, the X-rotation gate $\mathbf{Rx}(\beta)$ by angle β, the controlled Z-rotation gate (i), part of which is a Z-rotation gate $\mathbf{Rz}(\delta)$ by angle δ, and finally the **CNOT** gate. Taken from [36] and used with permission

hard task for classical computers. And on a physical, or operational, level, a quantum circuit is a set of instructions for a quantum computer to execute.

In the circuit model of quantum computation, qubits typically start at easily preparable states, $|0\rangle$, and then a sequence of gates is applied. Next, the qubits are read and the results are stored in standard digital memory, which is accessible for further handling. In practice, a quantum computer works alongside a classical computer, which in effect acts as the interface between the user and the quantum machine. The classical computer enables the user to handle the measurements for practical applications.

Once a circuit has been physically implemented on a quantum computer, and once it has been run, the qubits are measured. For an n qubit circuit this gives n bits. As a quantum computer is probabilistic, the circuit must be run many times to give statistics to estimate outcome probabilities. The design of a circuit must be such that it encodes the problem in a way that the outcome probabilities are obtained to give the answer to what one wishes to solve.

Building and running a quantum computer is an engineering task on a completely different scale from that of building and running a classical computer. Not only must qubits be shielded from random errors picked up from the environment, but one must also avoid unwanted interactions between multiple qubits within the same device. In order to avoid such problems, the number of successive operations on qubits (i.e., 'circuit depth') is limited on current quantum hardware. It is expected that to give an advantage for large scale problems one must have many fault-tolerant qubits, which are obtained by error correction techniques. These involve encoding the state of a 'logical' qubit in the state of many 'physical' qubits. As this is a difficult engineering task, quantum computers with fault-tolerant qubits are not available at the time of writing. Current machines are known as Noisy Intermediate-Scale Quantum (NISQ) computers and at the time of writing, they typically have less than 100 physical qubits.

13.5 DisCoCat Modelling

Development in the growing new field of QNLP is greatly facilitated by the Distributional Compositional Categorical (DisCoCat) modelling of natural language semantics. In DisCoCat, grammar dictates the composition of word meanings to derive the meaning of a whole sentence [13].

DisCoCat is a natural framework to develop natural language-like generative music systems with quantum computing. At the core of DisCoCat, as the model was originally formulated, is Joachim Lambek's, algebraic model of pregroup grammar; the curious reader may refer to [14] for a more rigorous mathematical treatment.

At the core of DisCoCat, the compositional structure of language is captured by (symmetric) monoidal categories, a.k.a. *process theories*. This mathematical formalism comes equipped with a formal graphical notation in the form of *string diagrams*. Meaning is encoded by a mapping, which endows semantics on the diagram rep-

resenting the grammatical structure of a text. This is in fact how DisCoCat was introduced in the first instance, by sending words to vectors, or in general, higher-order tensors, which are connected according to the grammatical dependencies to form a tensor network. Contracting the tensors along with the connections results in the derivation of the meaning of a text.

Categorical quantum mechanics (CQM) is a framework that reformulates quantum theory in terms of process theories. In this context, string diagrams describe quantum protocols involving quantum states, transformations, and measurements [1,12]. Low-level and fine-grained diagrammatic calculi, which build on the philosophy of CQM, such as ZX-calculus [11], are promising complementary—or even alternative—approaches to reasoning about quantum systems and processes. We see then that string diagrams provide a common language via which we can make the connection from language to quantum theory. After all, Hilbert spaces, which are where quantum states are encoded, are vector spaces. By analogy, many-body quantum states encode word meanings. And grammatical reductions correspond to processes such as quantum maps, quantum effects, and measurements.

To take advantage of this formal mathematical analogy, in order to perform grammar-based musical experiments on quantum processors, we employ DisCoPy [22], an open-source toolbox for manipulating string diagrams and implementing mappings to underlying semantics of our choosing. Among its features, DisCoPy includes tools for defining Context-Free Grammars (CFGs), Lambek's pregroup grammars [34], tensor networks [52], ZX-calculus, and other diagram-based reasoning processes, under the same compositional paradigm. In essence, and particular to this work, DisCoPy provides the means for mapping representations of musical systems into quantum circuits, encoding quantum processes, to be implemented on a quantum computer. For instance [38,39], used DisCoPy and Quantinuum's compiler t|ket⟩ [60] to develop and deploy a pioneering QNLP experiment on quantum hardware. Here the sentences were represented as parameterised quantum circuits and the parameters were trained to perform on a sentence-classification task. A larger-scale experiment of the same nature was reported by [36]. The models built for those experiments have been further developed into a Python library for QNLP known as `lambeq` [31], which is named in homage to Joachim Lambek. Indeed, part of the work reported in this chapter was conducted using an early version of `lambeq`.

13.5.1 A Musical DisCoCat Model

In pregroup grammar, the syntax of a sentence is captured by a finite product of words of different pregroup types. In our case, musical snippets will play the role of words, and musical compositions the role of sentences.

Let us define a musical composition σ as a finite product of musical snippets w as follows: $\sigma = \prod w_i$, where the product operation captures the sequential nature of the string representing a musical piece, in analogy with words being placed side by side in a sentence. Each snippet is assigned a pregroup type $t_w = \prod b_i^{k_i}$ comprising a product of basic types b_i from a set of types B. Basic types also have adjoints:

$k_i \in \{\ldots, ll, l, _, r, rr, \ldots\}$. Then, the type of a given musical composition σ is simply the product of the types of its snippets.

A composition is deemed valid, that is, grammatically correct—or musical—if and only if its type reduces to a special type $s \in B$. These reductions take place by means of an algebraic process of pair-wise annihilations of basic types and their adjoints according to the following rules: $b^l \cdot b \to 1$, $b \cdot b^r \to 1$, $b^{ll} \cdot b^l \to 1$, $b^r \cdot b^{rr} \to 1, \ldots$. To visualise this, let's go back to natural language and consider a sentence with a transitive verb: *Bob plays guitar*. Here, the transitive verb is a process that expects two input nouns of, say, type n, on each side, in order to output a sentence type s. The type of the transitive verb is therefore denoted as $n^r \cdot s \cdot n^l$. There exists a grammatical reduction following the algebraic rules of pregroup types:

$$n \cdot (n^r \cdot s \cdot n^l) \cdot n \to (n \cdot n^r) \cdot s \cdot (n^l \cdot n) \to 1 \cdot s \cdot 1 \to s \tag{13.1}$$

The DisCoCat string diagram for Eq. 13.1 is shown in Fig. 13.4. They carry a type. The wires bent in a U-shape, or cups, represent the reductions. Complex networks of processes and relationships can be designed by connecting boxes with input and output wires and observing that the types are respected and are reducible as required.

Such pregroup modelling is directly applicable to music. For instance, consider the example shown in Fig. 13.5. Here we defined a relationship $n^r \cdot s \cdot n^l$ between two musical notes (type n): C4 and A4. The relationship states that note A4 follows note C4 (Fig. 13.6). In this particular example, *sequence* is not a verb but an actual action to be carried out by some hypothetical generative music system.

Figure 13.7 depicts an example of a pregroup diagram representing a short musical composition, which is shown in Fig. 13.8. In this case, there are five musical snippets (labelled as p4, p9, p7 and p5), each of which comprises several musical notes forming two entire musical bars each.

The semantic map is a functor $\mathcal{F}$ that sends basic types (n, s) to vector spaces (N, S). Take the example in Fig. 13.4, 'Bob plays guitar'. The DisCoCat diagram can be decomposed as diagram $=$ (Bob $\otimes$ plays $\otimes$ guitar) $\circ$ cups. The second word 'plays' is a transitive verb and has the pregroup type $n^r \cdot s \cdot n^l$, whilst 'Bob' and 'guitar' have pregroup type n. The functor $\mathcal{F}$ sends the diagram to vector space semantics

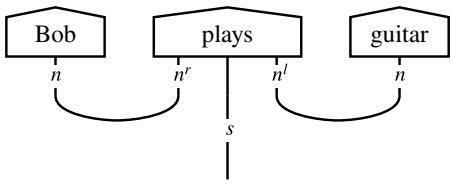


Fig. 13.4 Pregroup diagram for transitive the sentence 'Bob plays guitar'

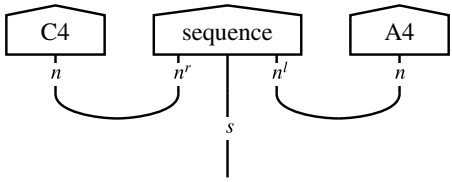


Fig. 13.5 Pregroup diagram for musical form shown in Fig. 13.6
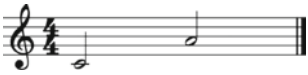


Fig. 13.6 A sequence of two notes: C4 and A4

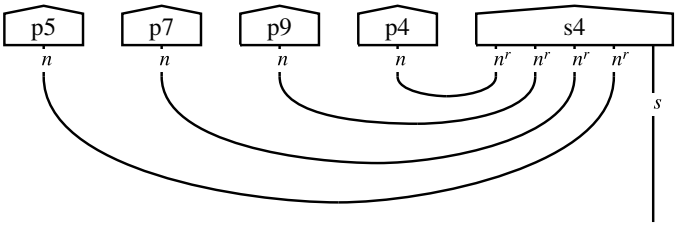


Fig. 13.7 Pregroup diagram for a musical composition form

by dropping the adjoints (l, r) and sending the pregroup type to the corresponding vector spaces. Based on their pregroup types, the words of the sentence will be sent to tensors of the corresponding vector space (Eq. 13.2).

$$\begin{aligned} \mathcal{F}(\text{Bob}), \mathcal{F}(\text{guitar}) \in \mathcal{F}(n) = N \\ \mathcal{F}(\text{plays}) \in \mathcal{F}(n^r \cdot s \cdot n^l) = N \otimes S \otimes N \end{aligned} \tag{13.2}$$

Moreover, $\mathcal{F}$ translates all reductions cups to tensor contractions. Conceptually, the functor can be thought of as something that performs tensor contraction between the words and the cups to return a vector for the whole sentence (Eq. 13.3). In Sect. 13.6.4, we will relay the details of the corresponding mapping to quantum processes.

$$\mathcal{F}(\text{diagram}) = (\mathcal{F}(\text{Bob}) \otimes \mathcal{F}(\text{plays}) \otimes \mathcal{F}(\text{guitar})) \circ \mathcal{F}(\text{cups}) \in S \tag{13.3}$$

Exactly the same principles are applicable to music. For instance, assuming the example in Fig. 13.5, and a set of musical notes N, the relationship *sequence* acquires the meaning $s = C4 \cdot R \cdot A4$ with $\{C4, A4\} \in N$ and $R \in (N \otimes S \otimes N)$.

13.6 Machine Learning of Music

In this section we introduce our DisCoCat music model and the quantum machine learning algorithm that we built to differentiate between two categories of music. Example code for the machine learning process can be found at [https://github.com/CQCL/Quanthoven]. A schematic overview of the algorithm is shown in Fig. 13.9.

In a nutshell, the algorithm learns from corpora generated by a bespoke context-free grammar (CFG). However, the quantum machine learning algorithm needs to see the structure of the compositions in the form of parameterised quantum circuits. Thus, the system needs to transform its underlying CFG structures into quantum circuits. This is done by converting CFGs into pregroups diagrams, which are then
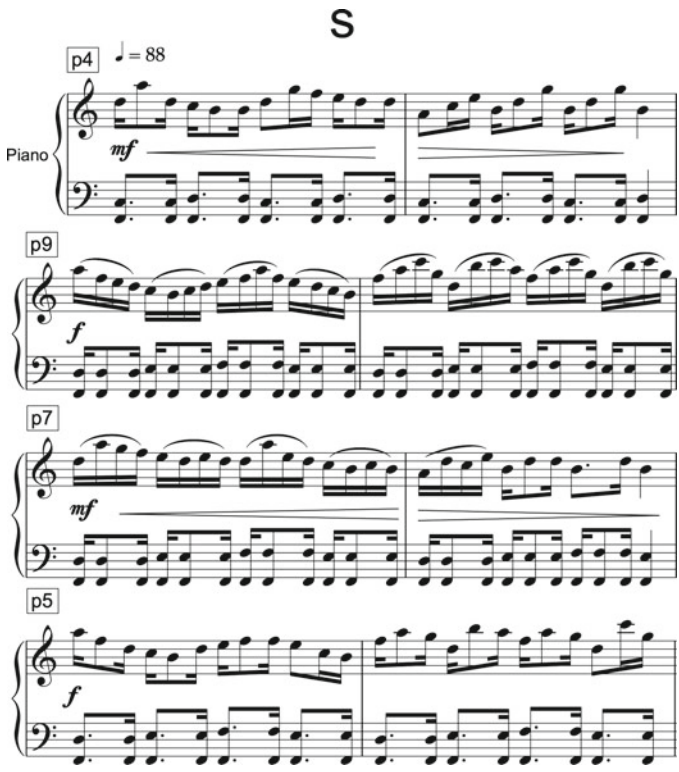


Fig. 13.8 A short musical composition corresponding to the pregroup diagram in Fig. 13.7

optimised before they are translated into quantum circuits. This optimisation is highly advisable because cups tend to produce more qubits than necessary when converting pregroup diagrams into a quantum circuit. The system then generates instructions for an appropriate quantum computer to run the circuit.

13.6.1 Generating a Training Corpus with a Context-Free Grammar

Here we define a Context-Free Grammar (CFG) to generate musical compositions for piano, which will be used to teach the machine learning algorithm. And *Quanthoven* will use this CFG to generate new pieces after it has learn to classify them.

The lexicon of our CFG contains four types of musical snippets. As it was mentioned already, a snippet is the equivalent of a word in natural language. Whereas a word is formed by letters from an alphabet, a snippet is formed by notes from a pitch framework.[4] Likewise, whilst combinations of words form sentences, combinations of snippets form musical sequences; that is, musical compositions.

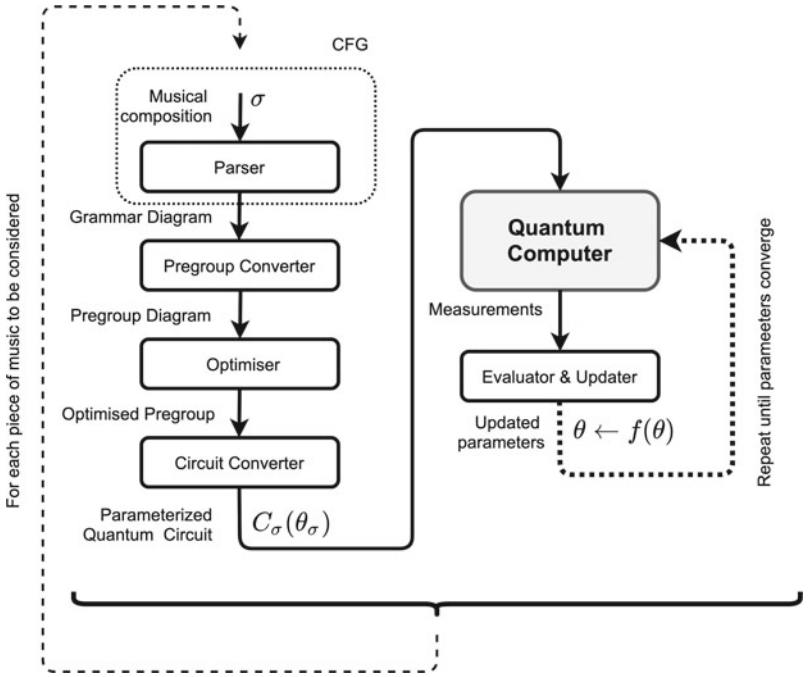


Fig. 13.9 Schematic overview of our quantum machine learning algorithm

As the second level of the Chomsky hierarchy, CFGs are expressive enough to model a vast portion of natural language. An informal, diagrammatic representation of a CFG is to draw words in the lexicon as boxes with one output wire. And production rules can be represented as boxes with multiple input wires and one output wire. Then, sequences can be built by freely composing these boxes. A more detailed explanation about combining CFGs with monoidal category theory can be found in Appendix A.1.

[4] In the context of this work, a musical pitch framework defines the notes that characterise a specific musical style or culture. An octave is defined by two notes, one having twice the pitch of another. Different pitch frameworks have their own way of subdividing the octave to create distinct scales of musical notes. Conventionally, there are 1,200 cents to the octave. Western European music normally subdivides the octave into 12 notes, equally spaced at 100 cents. This forms the so-called chromatic scale. However, some cultures subdivide the octave into notes with different spacings between one another. For example, Slendro Indonesian music subdivides the octave into five notes, spaced by 240 cents from each other. And most Middle Eastern music divides the octave into six notes spaced by 200 cents. Furthermore, no matter how an octave is divided, the notes can be played at different registers, or heights, depending on the instrument at hand. For instance, most pianos can produce 88 notes, accommodating seven octaves. That is, one can play seven different notes 'E' on the piano (or 'Mi' in traditional nomenclature). The snippets discussed in this article are made from the piano's 88 note-set.



Fig. 13.10 Diagrammatic representation of snippets

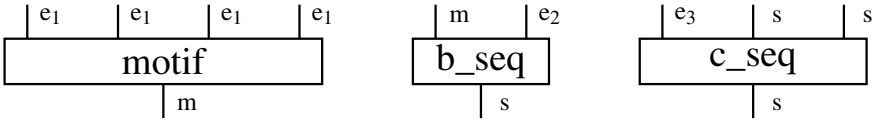


Fig. 13.11 Diagrammatic representation of musical sequences

Verbal languages have different *types* of words, such as nouns, verbs, adjectives, and so on. Grammatical rules then dictate how they are combined to form sentences. Similarly, our composition system contains four *types* of snippets: ground (g), primary (p), secondary (s) and tertiary (t) snippets. Some examples are shown in Fig. 13.12. These will be represented below as s, e_1, e_2 and e_3, respectively. We annotate the wires of the boxes with these types to enforce how the lexicon and production rules can be combined in a grammatical way to form musical compositions (Fig. 13.10).

Languages have different kinds of sentences; e.g., simple, compound, complex, and compound-complex sentence structures. By the same token, here we have three types of musical sequences: motif, basic (b_seq) and composite (c_seq) sequences , respectively (Fig. 13.11).

In a nutshell, a motif is composed of four primary snippets, and a basic sequence is composed of a motif followed by a secondary snippet. A composite sequence will always start with a tertiary snippet, followed by two elements. Each of them can be one of three options: a tertiary snippet, a basic sequence, or another composite one (Fig. 13.12).

Unlike natural languages, the productions rules of this musical grammar are defined explicitly. Therefore it is possible to implement a parser to recover the derivation from the sequence. In fact, our CFG is parsable by a LL(1) parser and hence is a LL(1) grammar [59]. In contrast, parsing natural language requires a contextual understanding of the words, so a statistical CCG parser [66] combined with a conversion from CCG to DisCoCat [65] is necessary. By explicitly implementing a parser, we show that the grammatical structure is recoverable from a sentence and need not be explicitly provided to the DisCoCat model: other models are welcome to take advantage of it.

Firstly, we generated a corpus of 100 compositions for piano. Then, we annotated the corpus according their meaning: *rhythmic* or *melodic*. As explained earlier, the overarching meaning of a composition describes an overall perceived property. Thus, compositions perceived as having prominent fast and loud rhythmic patterns were labelled as RIT (for rhythmic). Otherwise, those having above all successions of notes forming melodies and harmonies were labelled as MEL (for melodic).
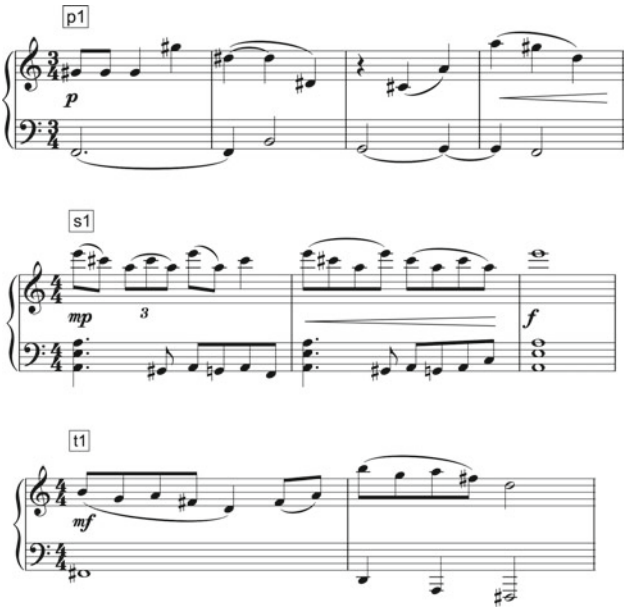


Fig. 13.12 Examples of primary, secondary and tertiary snippets, respectively

The annotation of the corpus was carried out manually. The labellings were agreed upon by the authors after they independently listened to the 100 compositions and reached a consensus. Equation 13.4 shows how the corpus Σ of annotated compositions σ look like. The symbols $t3$, $g1$, $p3$, and so on, represent the snippets that form a respective composition.

$$\begin{aligned}\Sigma = \{&(1, \text{MEL}, [t3,\ g1,\ p3,\ p1,\ p3,\ p3,\ s3]),\\ &(2, \text{RIT}, [p4,\ p9,\ p7,\ p5,\ s1]),\\ &(3, \text{RIT}, [t3,\ g2,\ g2]),\\ &...\}\end{aligned} \tag{13.4}$$

The complete lexicon of musical snippets is provided in Appendix A.2. Although some snippets are more rhythmic than others, and vice-versa, we made sure there is a considerable overlap of snippets between the two categories. This ensures that the machine learning task is realistic. After annotation, the corpus is split into training, development, and test sets with a $50 + 25 + 25$ split, respectively. The compositions last for different durations, ranging for a few seconds to various minutes. A symbolic representation of the dataset is available in Appendix A.3[5]

[5] Audio files are available at [https://github.com/CQCL/Quanthoven/]. Note, these were synthesised from MIDI codes.

13.6.2 Pregroup Converter: From Context-Free Grammars to Pregroup Grammars

In this section, we give a functorial mapping from musical CFGs to pregroups. The DisCoCat model is more naturally suitable for pregroup grammars as they are lexicalised grammars. In DisCoCat, most of the grammatical information is stored in the lexicon instead of the grammatical production rules. This is not the case for CFGs. Hence the need to convert to pregroups.

In natural language, a transitive verb is a word that takes a noun on the left and a noun on the right to give a sentence, and so has the pregroup type $F(VP_{TV}) = n^r \cdot s \cdot n^l$. An adjective is a word that takes a noun on the right to give another noun, and so has the pregroup type $F(Adj) = n \cdot n^l$.

Let us reason with analogies here: first we fix the types ground and primary elements, s and e_1, to be the atomic types for our pregroup grammar. This is done by setting $F(s) = s$ and $F(e_1) = n$. Since a motif is made of four primary elements, we give it the pregroup type $F(m) = n \cdot n \cdot n \cdot n$. A secondary element takes a motif on the left to give a sentence, it has the pregroup type $F(e_2) = F(m^r \cdot s) = F(e_1 \cdot e_1 \cdot e_1 \cdot e_1)^r \cdot s = n^r \cdot n^r \cdot n^r \cdot n^r \cdot s$. Finally, a tertiary element takes two sentences on the right to give another sentence, and so it has the pregroup type $F(e_3) = s \cdot s^l \cdot s^l$.

Once we have used our functor to convert the CFG types into the appropriate pregroup types, the CFG production rules become pregroup contractions, which are represented as 'cups' (Fig. 13.13). An example of this conversion on an actual musical grammar can be found in Fig. 13.14.

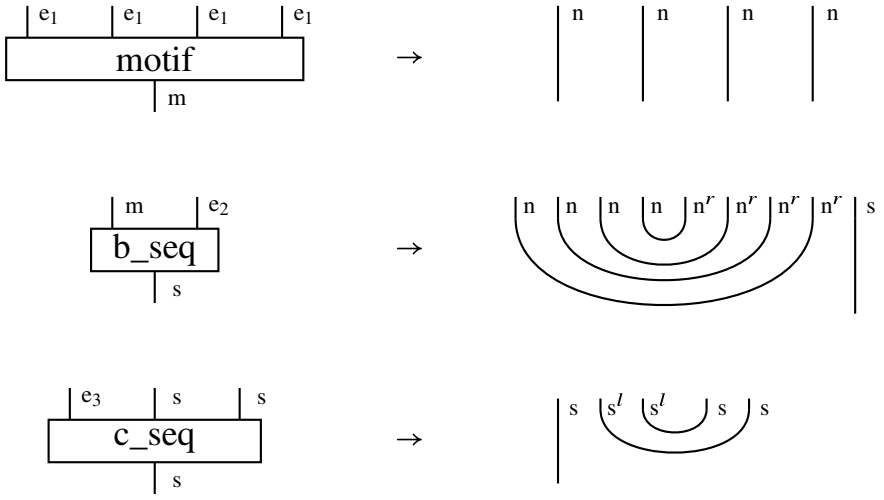


Fig. 13.13 Diagrammatic representation of pregroup contractions

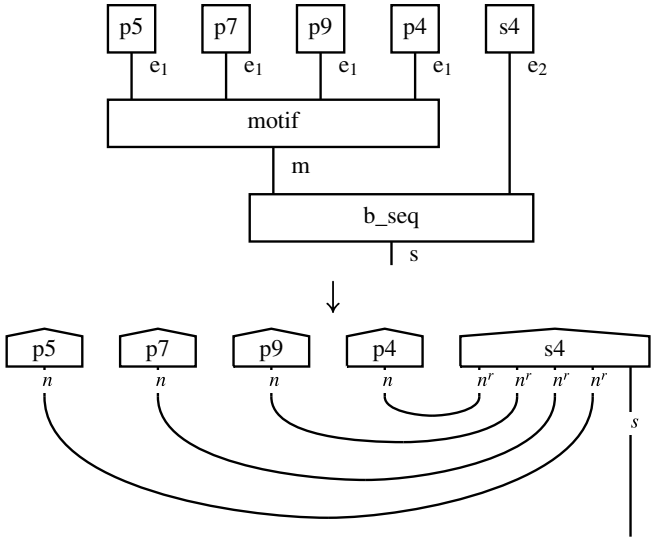


Fig. 13.14 Converting a CFG diagram into pregroup grammar

13.6.3 Optimiser: Diagrammatic Rewriting

Once a CFG diagram is converted into a pregroup grammar diagram, a circuit functor can be applied to transfer the pregroup diagram into a quantum circuit; this will be explained below, in Sect. 13.6.4. However, due the length of the compositions produced by our generation procedure, the resulting quantum circuit, after the functorial conversion has been applied, has too many qubits to be efficiently simulated before being executed on quantum hardware.

Furthermore, to perform the highly entangled qubit measurements—or Bell measurements—required by pregroup grammars, we need to use postselections. To perform a postselection on a qubit, one measures the qubit and then discards the circuits that do not have the desired outcome on the respective qubit. As a rough estimate, postselection halves the number of usable shots.[6] By performing n postselections, we reduce the number of usable shots by a factor of 2^n. For example, if a 12-qubit circuit requires 11 postselections, then we would expect approximately only 4 out of 8192 shots to be usable after postselection. This makes our results even more sensitive to the noise of a quantum computer.

One way to ameliorate both problems is to perform some diagrammatic rewriting. By rotating certain words in the pregroup diagram, we can eliminate the cups that correspond to Bell measurements using what we refer to as the snake equation. In terms of linear algebra, the rotation operation corresponds to taking the transpose. This rewriting technique, first used in [36], can also be applied to our musical pre-

[6] 'Shots' is a term used to refer to repeated runs of a circuit.

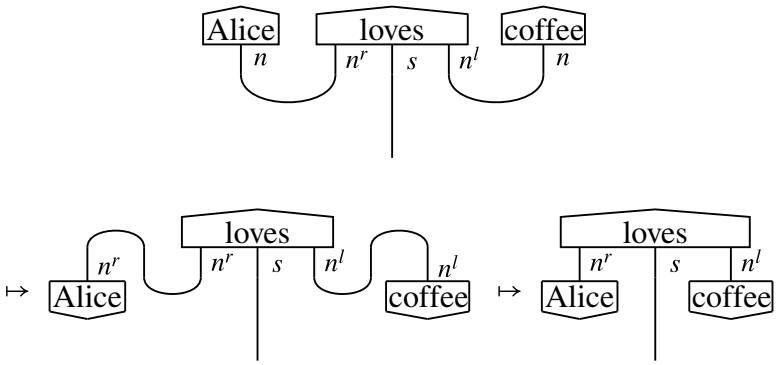


Fig. 13.15 Optimisation through rewriting

group diagrams. In Fig. 13.15, the number of postselections performed is halved from 8 to 4, so the number of useful shots has increased by a factor of 16.

By applying this rewriting technique to our diagrams, we reduce the worse-case circuit size in our training set from 25 qubits to 13 qubits. This is a reduction of 12 postselections. Thus, the number of shots required is reduced by a factor of 4096.

13.6.4 Circuit Converter: Translating Musical Compositions into Quantum Circuits

With DisCoCat, a string diagram for a musical composition σ (which is generated by a CFG and thus has a corresponding pregroup diagram) can be translated onto a parameterised quantum circuit $C_\sigma(\theta_\sigma)$ over a parameter set θ_σ. The translation assigns a number of qubits q_b to each wire carrying a type b. Therefore, Hilbert spaces are defined on the wires (Fig. 13.16).

The snippet-state of the η-th snippet in a string diagram is reinterpreted as a pure quantum state prepared from a given reference product-state by a circuit C_η (Eq. 13.5).

$$C_\eta(\theta_\eta)\,|0\rangle^{\otimes q_\eta} \;,\quad q_\eta = \sum_{i=1}^{|\eta|} q_{b_i} \tag{13.5}$$

The width of a snippet-circuit depends on the number of qubits assigned to each of the $|\eta|$-many basic types $b \in B$ assigned to the snippet. Given a composition σ, the system firstly concatenates the snippet-states of the snippets as they appear in the composition. Concatenation corresponds to taking their tensor product (Eq. 13.6).

$$C_\sigma(\theta_\sigma)\,|0\rangle^{\otimes q_\sigma} = \bigotimes\nolimits_{\eta_j} C_{\eta_j}(\theta_{\eta_j})\,|0\rangle^{\otimes q_{\eta_j}} \tag{13.6}$$

The circuit in Fig. 13.17 prepares the state $|\sigma(\theta_\sigma)\rangle$ from the all-zeroes basis states. A musical composition is parameterised by the concatenation of the parameters of its snippets: $\theta_\sigma = \cup_{\eta\in\sigma}\theta_\eta$, where θ_η defines the snippet-embedding $|\eta(\theta_\eta)\rangle$.

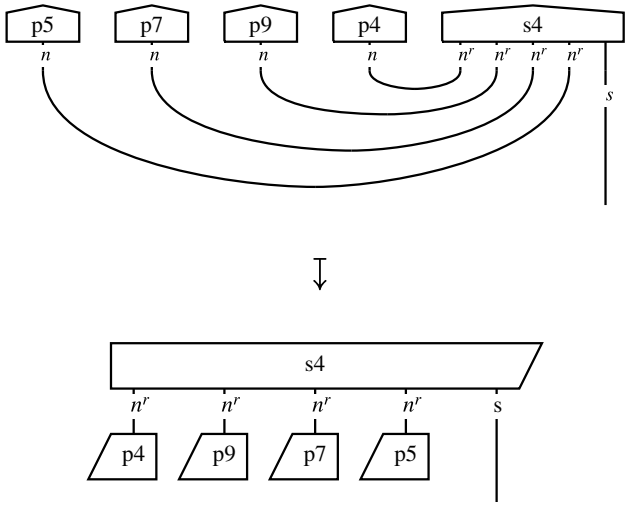


Fig. 13.16 The pregroup diagram in Fig. 13.14 rewritten by removing the caps

Then, Bell effects are applied, as determined by the cups representing pregroup reductions. Cups are interpreted as entangling circuits that implement a Bell effect by postselecting on a Bell state. This effect of grammar, denoted g, acts on the state of the composition and returns the state $g(|\sigma(\theta_\sigma)\rangle)$.

13.6.5 Training the System to Classify

Here we describe how we trained the system with the aforementioned musical dataset to distinguish between two categories of music: rhythmic and melodic.

As we are performing binary classification, we choose to represent the output of the system in one-hot encoding. This means that our two class labels are [0, 1] and [1, 0] corresponding to 'melodic' and 'rhythmic' music respectively. In the DisCoCat model, this can be conveniently achieved by setting the sentence dimension space (recall the whole circuit is represented by an open sentence wire) of our DisCoCat model to one qubit. The circuit C_σ for each composition σ is evaluated for the current set of parameters θ_σ on the quantum computer giving output state $|C(\theta_\sigma)\rangle$. The expected prediction $L^i_{\text{pred}}(\sigma, \theta)$ is given by the Born rule in Eq. 13.7, where $i \in \{0, 1\}$ and $\theta = \cup_{\sigma\in\Sigma}\theta_\sigma$.

$$L^i_{\text{pred}}(\sigma, \theta) := |\langle i|C_\sigma(\theta_\sigma)\rangle|^2 \tag{13.7}$$

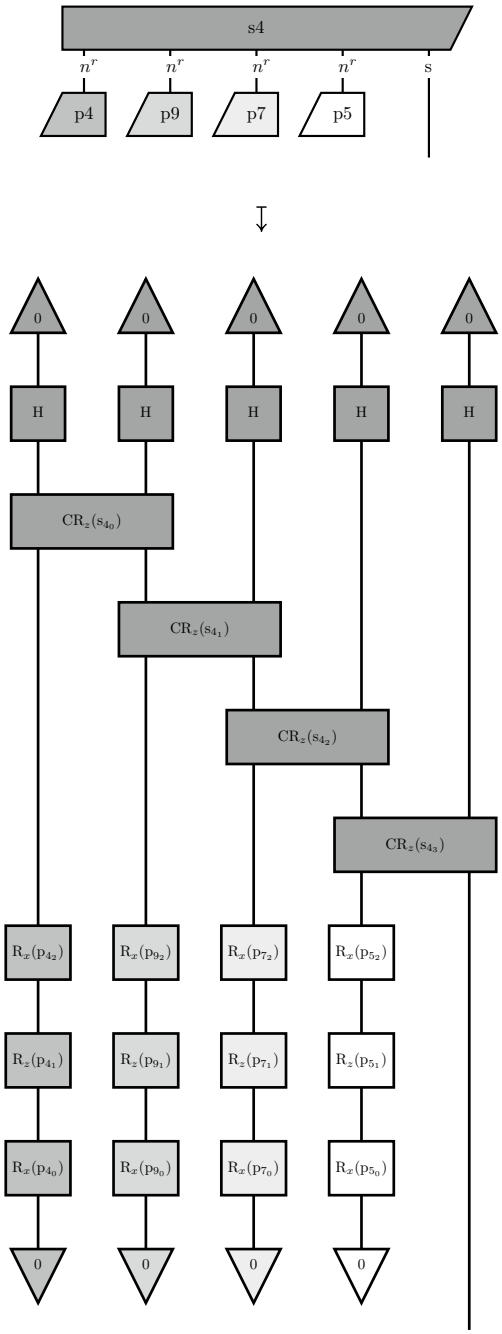


Fig. 13.17 The pregroup diagram in Fig. 13.16 converted to a quantum circuit, according to the IQP and Euler decomposition ansätze. Regions of the diagram and circuit have been coloured to illustrate the transformation of each part of the diagram into a quantum circuit

By running and measuring the outcome (either [0, 1] or [1, 0]) of the circuit many times, the average outcome will converge towards this value. Then $L^i_{\text{pred}}(\sigma, \theta)$ is normalised to obtain a smooth probability distribution (Eq. 13.8).

$$l^i_{\text{pred}}(\sigma, \theta) := \frac{L^i_{\text{pred}}(\sigma, \theta) + \epsilon}{\sum_j \left(L^j_{\text{pred}}(\sigma, \theta) + \epsilon \right)} \tag{13.8}$$

The smoothing term is chosen to be $\epsilon = 10^{-9}$. The predicted label is obtained from the probability distribution by setting a decision threshold t on $l^0_{\text{pred}}(\sigma, \theta)$: if $t < l^0_{\text{pred}}(\sigma, \theta)$ then the predicted label is [0, 1]. And if $t \geq l^0_{\text{pred}}(\sigma, \theta)$ then the predicted label is [1, 0]. This threshold can be adjusted depending on the desired sensitivity and specificity of the system. For this chapter, the threshold $t = 0.5$ is selected.

To predict the label for a given snippet well, we need to use optimisation methods to find optimal parameters for our model. We begin by comparing the predicted label with the training label using a loss function. Since we are performing a classification task, the binary cross-entropy loss function is chosen (Eq. 13.9).

$$\text{BCE}(\theta) = \sum_{\sigma} \sum_{i \in \{0,1\}} l^i_{\text{label}}(\sigma, \theta) \log(l^i_{\text{pred}}(\sigma, \theta)) \tag{13.9}$$

A non-gradient based optimisation algorithm known as SPSA (Simultaneous Perturbation Stochastic Approximation) is used to minimise our loss function [61]. Alternatively, one could use gradient-based optimisation algorithms by differentiating the DisCoCat diagram [63]. However, SPSA was found to be sufficient for our purposes.

In order to minimise the loss function, the system learns to classify the compositions by adjusting the parameters of the musical snippets. Philosophically, this is in contrast to typical machine learning models—that is, neural networks—where the weights of the network are adjusted rather than the weights of the word embeddings. This is because pregroup grammars are lexical. And in lexical grammar, whilst words have parameters that can be changed, the grammatical structure cannot be changed.

Unfortunately, the training process cannot be performed entirely on a real quantum device at the time of writing. Access to quantum hardware still is limited, due to the demand for and the limited number of available quantum processors at the time of writing. This can be problematic for running variational tasks where circuits must be run for many iterations in order to get satisfactory results.

Therefore, we firstly pre-train the model parameters using the exact tensor contraction of the quantum circuits on a classical computer. We use Python's JAX library[7] for just-in-time compilation, which speeds up the pre-training. This speed-up allows us to efficiently perform grid search and find the best hyper-parameter settings for the SPSA optimiser. The results for such a classical simulation are shown in Fig. 13.18 for the final settings chosen for the subsequent training.

[7] https://jax.readthedocs.io/en/latest/.

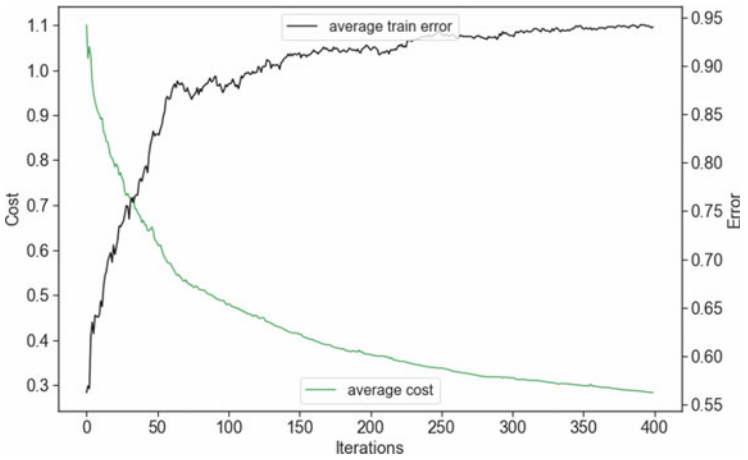


Fig. 13.18 Average over 20 different runs of a classical simulation of the training set showing the cost (green, left hand side axis) and the training error (black, right hand side axis)

Then, we simulate the quantum process on a classical machine. In order to do this, we need to take into account the statistical nature of experiments, which would involve running the same circuit thousands of times to obtain measurement statistics. And we also need to bear in mind the noise that is present in a real device. The parameters learnt from the exact pre-training are transferred to a model, which closely resembles the real quantum device, which includes noise simulation. This will ultimately improve testing performance on the real quantum device. So, the same circuit is simulated 8192 times with a noise model specific to the quantum device that will be used for testing. This process is carried out for each composition in the training set, for a chosen number of iterations. The train and development set results from the quantum simulations are shown in Fig. 13.19.

Once the pre-training phase is complete, and we are happy with the simulated results on the development set, then we take the parameters from the noisy quantum simulation and evaluate the test set on a real quantum computer. We used IBM Quantum's device `ibmq_guadalupe`, which is a 16-qubit superconducting processor with a quantum volume of 32. Despite the expected differences between a real experiment and a simulation, we can see that the model learns to classify compositions in the test set with an impressive final accuracy of 76%, as shown in Fig. 13.19.

13.6.5.1 Further Classification Tests

The results from the testing with quantum hardware (Fig. 13.19) exceeded our expectations. We conducted additional tests to probe the system further, which confirmed the system's positive performance.

For the new tests we ran our CFG to produce two additional datasets: one containing 60 compositions and another containing 90. As with the previous dataset, we

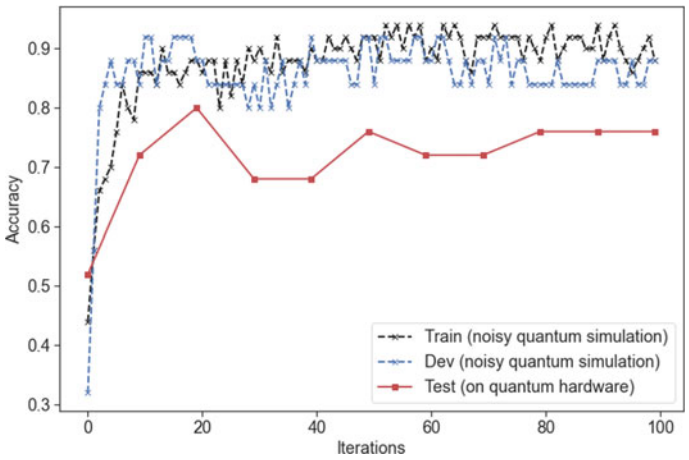


Fig. 13.19 Noisy quantum simulation to train the model (black), noisy quantum simulation of the development set (blue) and the test set run on on quantum hardware (red). The noise model used for the noisy simulation was that of the `ibmq_guadalupe` device, which is the hardware that we subsequently usedfor running the test set

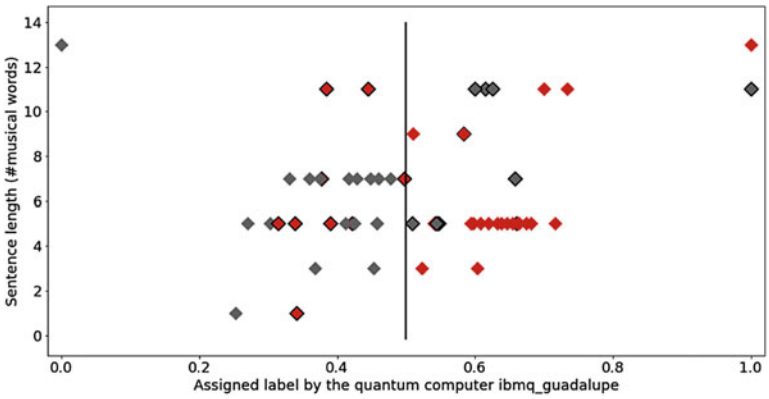


Fig. 13.20 Quantum computing classification of dataset with 60 compositions

auditioned and annotated the pieces manually. Again, both sets contain compositions of varying lengths. Then, we submitted the sets to the quantum classifier. Now we display the results from a different perspective than in Fig. 13.19.

Figure 13.20 shows that the system correctly classified 39 compositions from the set of 60. Melodic pieces correspond to a value plotted on the first element in the tuple of <0.5 (grey), whereas the Rhythmic ones correspond to a value ≥ 0.5 (red). Misclassified compositions have a black outline around them, as well as being the wrong colour for the side of the graph that they appear on. For clarity, Fig. 13.21 plots only those compositions that were classified correctly.

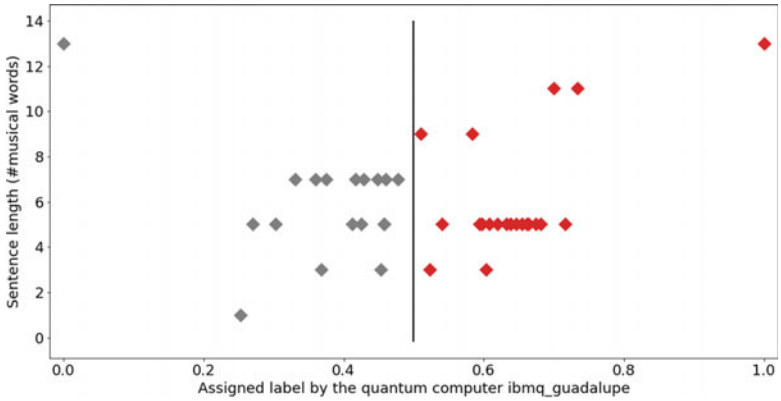


Fig. 13.21 A version of Fig. 13.20 plotting only those pieces that were classified correctly

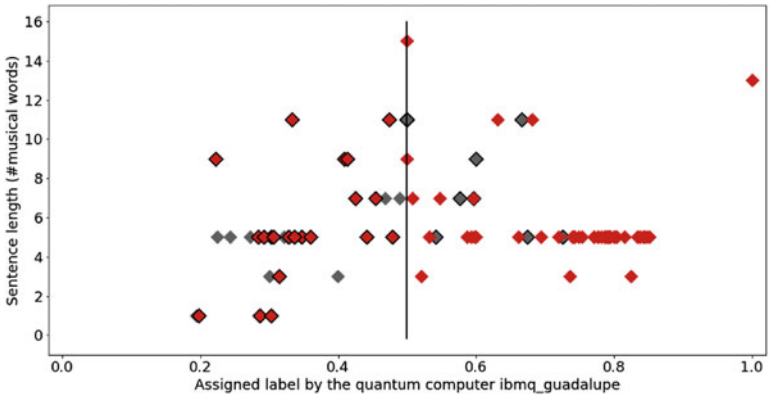


Fig. 13.22 Quantum computing classification of dataset with 60 compositions

Next, Fig. 13.22 shows that the system correctly classified 59 compositions from the set of 90. Again, Melodic (MEL) pieces corresponds to a value plotted on the first element in the tuple of <0.5 (grey), whereas the Rhythmic (RIT) ones corresponds to a value ≥ 0.5 (red). And Fig. 13.23 shows the same results, but plots only those compositions that were classified correctly.

13.7 *Quanthoven*: Leveraging the Quantum Classifier to Compose

Composer Lejaren Hiller, who also was a chemistry teacher, allegedly is a pioneer of programming computers to generate music. In 1957, he teamed up with Leonard Isaacson, also a chemist and composer, to program the ILLIAC I

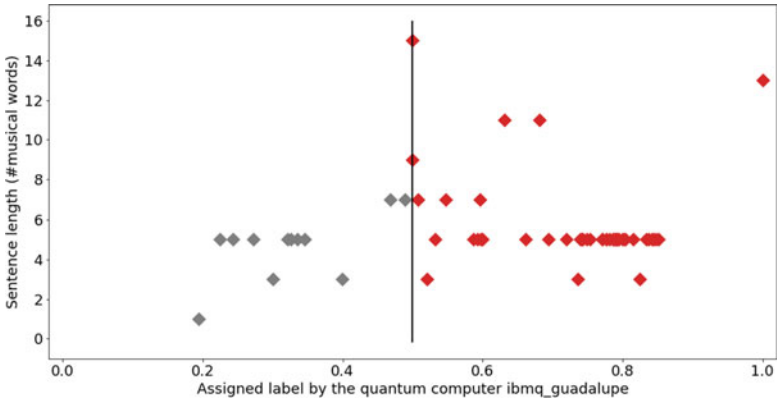


Fig. 13.23 A version of Fig. 13.22 plotting only those pieces that were classified correctly

(Illinois Automatic Computer)[8] at the University of Illinois, USA, to compose music. The computer produced materials for their string quartet, entitled *Illiac Suite*.[9]

In their initial experiments, they programmed the computer to do two things: (a) generate musical data[10] (pseudo-randomly), and (b) test if the data satisfied musical rules. Musical data that did not satisfy the rules were discarded. Conversely, those that satisfied the rules were kept for the ongoing composition. For instance, for the first movement of *Illiac Suite* they set the machine with rules from the famous treatise *Gradus ad Parnassum*, written by Joseph Fux in the 17th century [37].

Hiller's approach became an archetype for algorithmic music composition with computers, which is often referred to as the *generate-and-test* approach (Fig. 13.24). Various variants have been implemented, with increasingly sophisticated methods for generating musical data and probing them [20,41]. Such systems might generate notes that are checked one at a time, or generate phrases, larger sequences or even entire pieces. Human approval often takes place as well, whereby composers may further discard or amend computer-validated materials.

The generate-and-test approach also informed the development of Artificial Intelligence (AI) techniques for musical composition [23,46], including highly praised constraint-satisfaction methods [3]. Moreover, the generate-and-test method epitomises Evolutionary Computing techniques for music composition; e.g., using genetic algorithms [47]. Such systems generate new sequences using biologically inspired processes (e.g., genetic mating, reproduction and mutation) and then employ fitness measurements to evaluate them [43].

[8] This was the first von Neumann architecture computer built and owned by a university in the USA. It was put into service on September 22, 1952.

[9] Later retitled as "String Quartet No. 4".

[10] That is, sequences of symbols representing notes, rhythms and expressions such as 'pizzicato and 'arco', and so on.

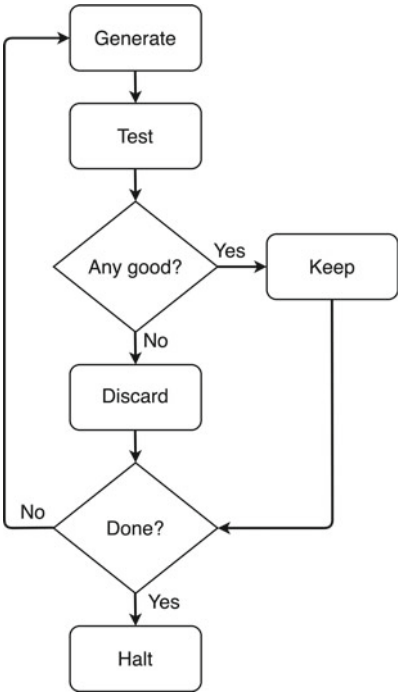


Fig. 13.24 Archetypal generate-and-test approach to algorithmic music composition

The quantum machine learning paradigm discussed above fits the generate-and-test approach to algorithmic music composition very nicely. To test this, we developed *Quanthoven*, a system for composition with the classifier presented above acting as the evaluator. In a nutshell, the CFG (Sect. 13.6.1) generates compositions and the trained classifier (Sect. 13.6.5) evaluates them (Fig. 13.24).

In the introduction, we mentioned that our ambition is to develop systems to aid musicians to compose 'music for Alice's tea party', or a 'tune to make Bob feel energetic'. In order to do this, the machine is required to handle the meaning of music; that is, the perceived properties of musical compositions, which holistically characterise their style. Our proof-of-concept gives a good hint about how this can be achieved.

Effectively, our machine learning algorithm learned what we mean by 'melodic' and 'rhythmic' music. The CFG can generate pieces with snippets combined in ways that may render a piece more melodic than rhythmic, or vice-versa. Some might be neither to our ears, others might be both. But the quantum classifier is able to suggest whether a CFG-generated piece is one or the other.

As a demonstration, we set ourselves the task of composing four pieces of music with *Quanthoven*: two for relaxing at a tea party and two aimed at inducing excitement. We set *Quanthoven* to generate two dozen pieces and save four, two for each category. The others were discarded on the spot; we do not even know what they sounded like.
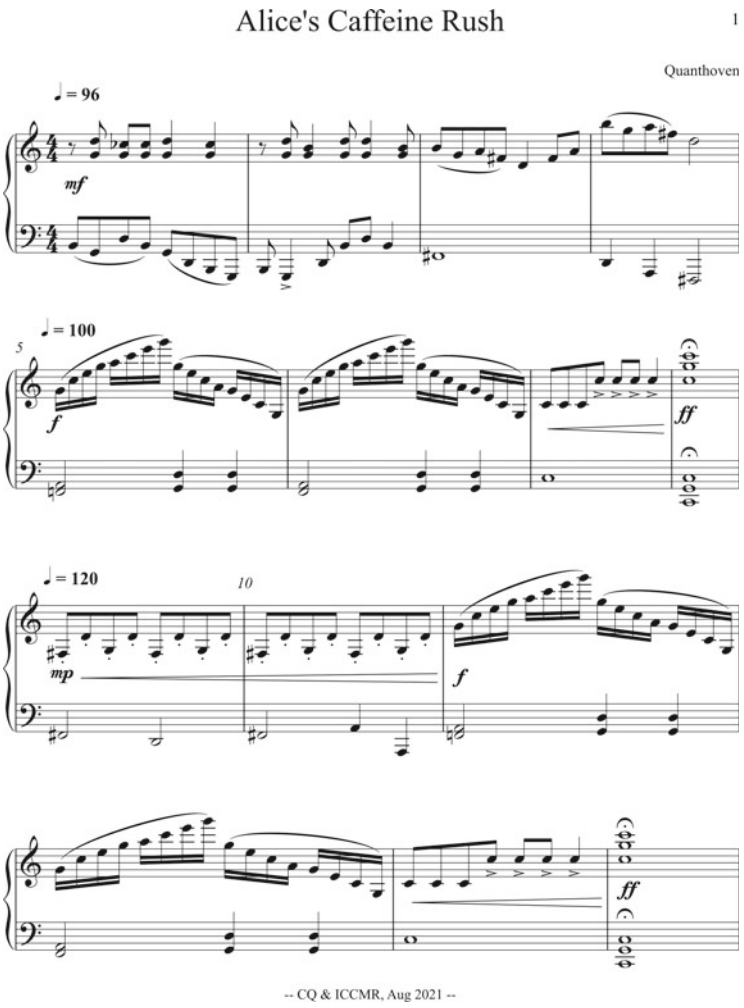


Fig. 13.25 First page of the composition *Alice's Caffeine Rush*

The saved pieces were encoded as MIDI files. Then, we uploaded these files into a music editor, formatted them, added titles and other details, and printed the scores. We hired a professional pianist to record them. The two pieces for relaxing at a tea party were entitled *Bob's Chamomile Slowdown* and *Alice's Mushroom Trip*. The ones aimed at making one feel excited were entitled *Bob's Cigar Buzz* and *Alice's Caffeine Rush*.

The score for *Alice's Caffeine Rush* is shown in Figs. 13.25 and 13.26. The eyes of a trained musician would immediately see patterns that characterise this piece as rhythmic, in particular, the section starting at bar 15. The scores for the other three pieces are provided in Appendix A.4.
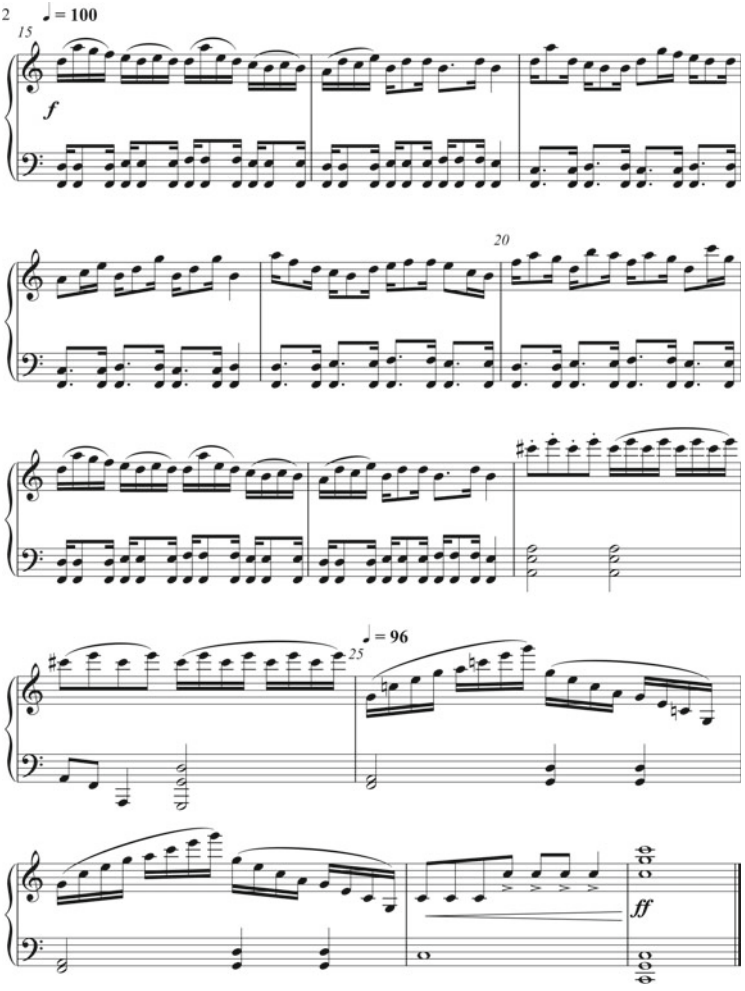


Fig. 13.26 Second page of the composition *Alice's Caffeine Rush*

13.8 Final Remarks

Recordings of all four pieces are released in SoundClick [58]. You are invited to listen to the compositions and make your own mind whether you agree with *Quanthoven* or not. We are rather satisfied with the results. But of course, they are debatable.

This chapter started by saying that people have different tastes and opinions about music. And it is also noted that people perceive and react to music differently from each other. However, the five authors agreed on the 'meanings' here. And the aim was to teach the machine to classify the pieces according to our opinions. Objectively, the system does what it says on the tin.

We made the conscious decision of conducting this experiment with music generated from a CFG designed from the ground up, completely from scratch. Of course, we could have written a piece of software to extract a CFG from given corpora of musical scores; e.g., [6]. Ultimately, we would extract this information directly from audio recordings. However, we wanted to test our approach with something other than existing styles of music. Moreover, we wanted to ensure that, at this stage of our research, the two categories in question share the same grammatical structure. Effective annotation methods for a much larger database should also be developed; for instance, based on information extracted from social media.

As quantum computing hardware and error-correction technologies evolve, we will certainly continue pushing the boundaries in tandem, with compositions of increased sophistication, more instruments, longer durations, more categories (or 'meanings'), and so on. These will probably require generative mechanisms other than CFG, something along the lines of transformational-generative grammars. Also, an important next step would be to design an actual quantum generative music engine drawing directly from the classifier, as opposed to using the classifier as a filter.

Acknowledgements This project was developed during Miranda's research residency at Quantinuum (formerly Cambridge Quantum) in 2021, which was partially funded by the QuTune Project. QuTune kick-started in the Spring of 2021 thanks to funding kindly provided by the UK National Quantum Technologies Programme's QCS Hub: [https://iccmr-quantum.github.io/]. Coecke acknowledges support from Foundational Questions Institute (FQXi). Project entitled *Categorical Theories of Consciousness: Bridging Neuroscience and Fundamental Physics*: [https://fqxi.org]. The authors thank pianist, Lauryna Sableveciute, and recording engineers, John Lowndes and Manoli Moriaty, at Liverpool Hope University, for the recordings of *Bob's Chamomile Slowdown*, *Alice's Mushroom Trip*, *Bob's Cigar Buzz*, and *Alice's Caffeine Rush*.

A Appendices

A.1 Context-Free Grammars and Monoidal Category Theory

A context-free grammar (CFG) is typically defined with a set of terminal words T, non-terminal symbols N, and a set of production rules of the form $\mathrm{A} \to \alpha$, where $\mathrm{A} \in N$ is a non-terminal symbol and $\alpha \in (N \cup T)^*$ is a string of terminal words and non-terminal characters. Any sentence that can be produced by freely applying the production rules on the starting terminal symbol S is considered to be a grammatical sentence.

For example, below is a CFG which defines a collection of sentences:

$$\mathrm{NP} \to \mathrm{Adj\ NP},\ \mathrm{S} \to \mathrm{NP\ V\ NP},\ \mathrm{NP} \to \mathrm{N}$$

$$\mathrm{N} \to n \in \{Alice, tea\}$$

$$\mathrm{V} \to v \in \{drinks, likes, hates\}$$

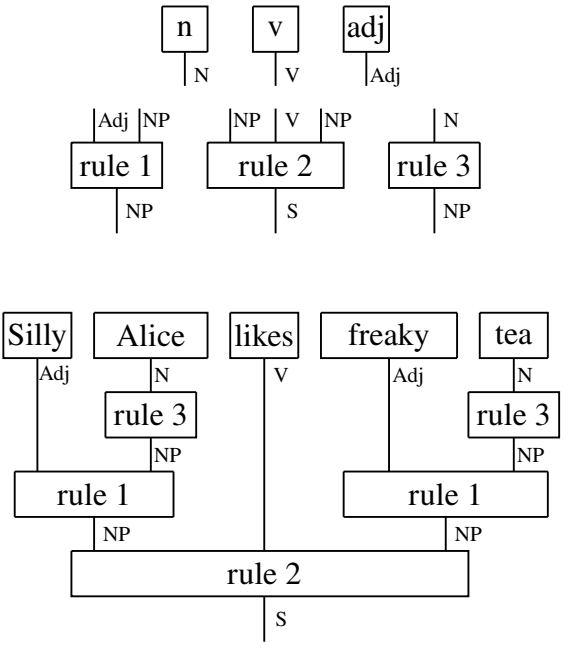


Fig. 13.27 A diagrammatic representation of the CFG example (top) and a sentence built using it (bottom)

$$\text{Adj} \to adj \in \{funny, silly, freaky\}$$

The general framework of CFGs can be thought of as a freely generated monoidal category. In diagrammatic notation, a monoidal category consists of boxes with input and output wires. The 'types' of the wires are the generating objects of the monoidal category. Wires of matching types can be connected by extending the wire vertically.

In a free monoidal category, any diagram produced by freely composing the boxes is a valid diagram in the category. This resembles how CFGs allow free applications of the production rules.

Production rules in a CFG of the form A $\to$ α are represented as a box with multiple input wires representing the string α and output wire representing the non-terminal A. Input wires representing terminal words in α are closed with boxes of type $I \to T$, where I is an auxiliary type for terminal types. For example, the CFG above can be described by freely composing the boxes in Fig. 13.27.

A.2 Lexicon of Musical Snippets

The lexicon for the music composition system presented in this chapter contains four types of snippets: ground (g), primary (p) secondary (s) and tertiary (t) snippets. Here, snippets are for music what words are for natural languages; languages have different types of words, such as nouns, verbs, and adjectives (Figs. 13.28, 13.29, 13.30 and 13.31).
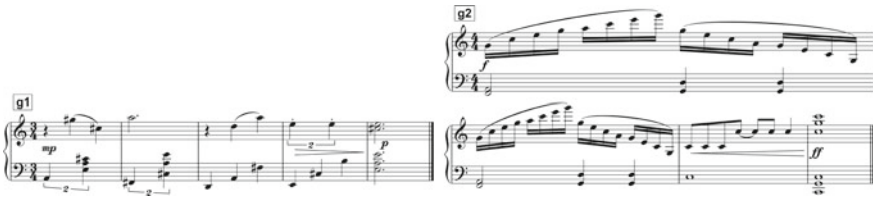


Fig. 13.28 Ground snippets g1 and g2
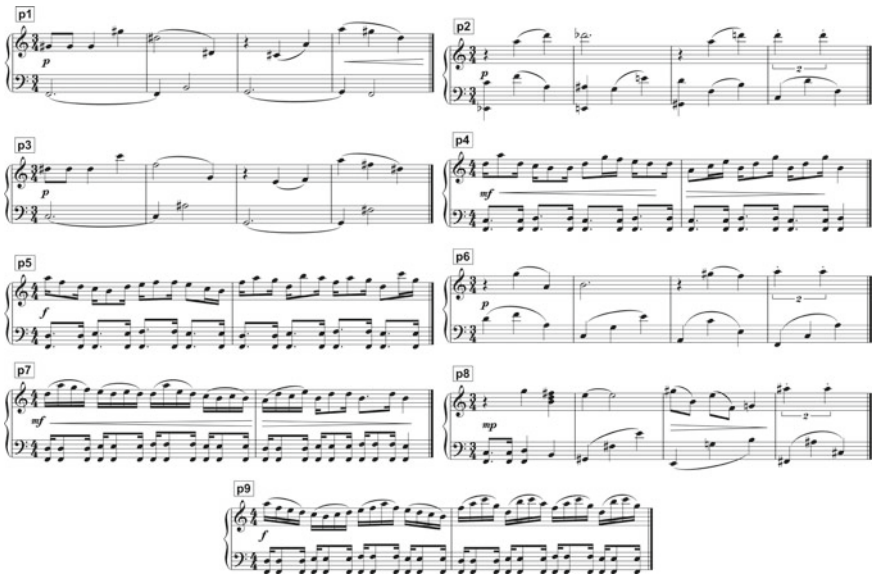


Fig. 13.29 Primary snippets p1, p2, p3, p4, p5, p6, p7, p8, and p9
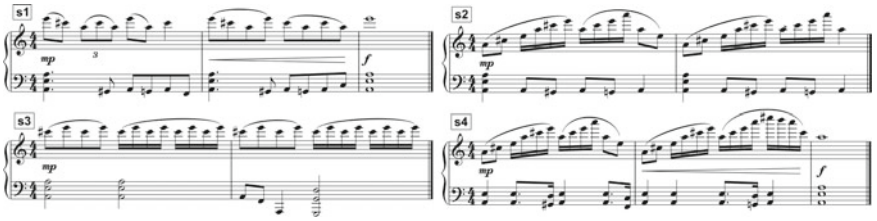


Fig. 13.30 Secondary snippets s1, s2, s3, and s4
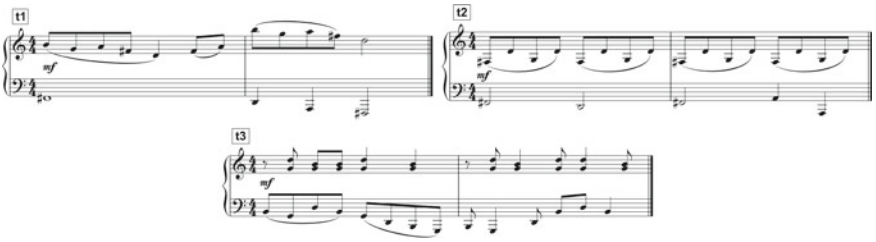


Fig. 13.31 Tertiary snippets t1, t2 and t3

A.3 Musical Datasets

A.3.1 Training Data

$\Sigma_\alpha = \{$(1, MEL, [t3, g1, g1]),
(2, MEL, [t3, p8, p1, p8, p1, s4, g1]),
(3, RIT, [t3, p9, p9, p5, p9, s4, g2]),
(4, RIT, [p9, p4, p4, p4, s3]),
(5, RIT, [p4, p9, p9, p9, s4]),
(6, RIT, [p5, p9, p4, p5, s1]),
(7, RIT, [t3, p9, p5, p7, p7, s1, g2]),
(8, MEL, [p1, p8, p2, p3, s4]),
(9, MEL, [p6, p1, p8, p3, s4]),
(10, MEL, [t2, g1, t1, g1, g1]),
(11, RIT, [p4, p5, p5, p7, s4]),
(12, MEL, [t2, t3, t2, g1, g1, g1, g1]),
(13, MEL, [p8, p1, p6, p2, s4]),
(14, RIT, [p5, p7, p7, p9, s2]),
(15, RIT, [p5, p7, p5, p9, s4]),
(16, RIT, [t3, g2, p9, p9, p7, p7, s1]),
(17, MEL, [p6, p2, p3, p2, s1]),
(18, MEL, [t1, g1, p3, p1, p1, p1, s2]),
(19, RIT, [p7, p7, p5, p7, s1]),
(20, RIT, [t3, p5, p9, p5, p7, s2, g2]),
(21, RIT, [p7, p4, p4, p9, s2]),
(22, RIT, [p5, p9, p5, p4, s1]),
(23, RIT, [p9, p9, p7, p9, s1]),
(24, RIT, [p7, p7, p9, p7, s1]),
(25, RIT, [p9, p4, p5, p7, s4]),
(26, MEL, [t3, p8, p6, p1, p2, s2, g1]),
(27, MEL, [t2, p6, p8, p8, p8, s1, g1]),
(28, MEL, [p6, p3, p6, p6, s2]),
(29, MEL, [p6, p8, p3, p1, s2]),
(30, MEL, [p1, p2, p6, p6, s4]),
(31, MEL, [p6, p1, p2, p8, s3]),

(32, MEL, [p1, p1, p6, p3, s2]),
(33, RIT, [p4, p9, p7, p5, s1]),
(34, RIT, [t2, g2, p5, p4, p5, p4, s1]),
(35, RIT, [p9, p9, p9, p5, s3]),
(36, RIT, [p9, p9, p5, p5, s4]),
(37, MEL, [p6, p2, p6, p2, s1]),
(38, RIT, [p5, p5, p5, p5, s4]),
(39, MEL, [p6, p1, p1, p1, s1]),
(40, RIT, [p5, p7, p5, p4, s2]),
(41, MEL, [t2, g1, p1, p6, p2, p6, s2]),
(42, MEL, [p1, p6, p3, p8, s2]),
(43, RIT, [p4, p5, p7, p7, s1]),
(44, MEL, [p1, p6, p8, p1, s2]),
(45, MEL, [p6, p1, p2, p3, s1]),
(46, RIT, [t2, g2, t1, g2, p9, p5, p4, p9, s2]),
(47, MEL, [t3, t1, g1, g1, p8, p6, p6, p8, s4]),
(48, MEL, [p2, p3, p8, p6, s2]),
(49, RIT, [t1, g2, t1, p7, p4, p7, p7, s4, g2]),
(50, RIT, [t1, t2, g2, p4, p9, p4, p7, s4, g2])}

A.3.2 Development Data

Σ_β = {(51, MEL, [t1, g1, p8, p6, p8, p2, s4]),
(52, MEL, [t1, p6, p8, p2, p8, s4, g1]),
(53, RIT, [p4, p5, p7, p4, s2]),
(54, MEL, [t2, p2, p8, p1, p6, s2, g1]),
(55, MEL, [p8, p2, p3, p1, s2]),
(56, MEL, [t3, p6, p2, p8, p2, s3, g1]),
(57, RIT, [p4, p9, p4, p9, s1]),
(58, RIT, [p7, p9, p5, p4, s4]),
(59, RIT, [t2, p7, p7, p4, p9, s1, g2]),
(60, RIT, [p9, p9, p7, p5, s4]),
(61, RIT, [p5, p9, p5, p7, s1]),
(62, RIT, [p4, p5, p4, p9, s2]),
(63, RIT, [p9, p4, p9, p9, s4]),
(64, RIT, [p4, p7, p5, p5, s3]),
(65, MEL, [p2, p1, p2, p2, s3]),
(66, RIT, [p7, p7, p9, p7, s4]),
(67, MEL, [p6, p3, p6, p6, s4]),
(68, RIT, [p9, p7, p9, p4, s3]),
(69, RIT, [p9, p9, p4, p5, s2]),
(70, MEL, [p8, p3, p2, p6, s2]),
(71, RIT, [p9, p4, p5, p4, s4]),
(72, MEL, [p6, p6, p2, p3, s4]),
(73, RIT, [p7, p7, p4, p5, s2]),

(74, MEL, [t3, g1, t3, g1, p1, p2, p2, p3, s2]),
(75, MEL, [t1, g1, g1])}

A.3.3 Testing Data

$\Sigma_\gamma = \{$(76, RIT, [p9, p9, p9, p9, s1]),
(77, MEL, [p3, p6, p8, p2, s4]),
(78, RIT, [p4, p7, p5, p9, s1]),
(79, RIT, [p9, p9, p5, p5, s3]),
(80, RIT, [p7, p9, p9, p7, s2]),
(81, RIT, [t3, g2, p4, p4, p9, p9, s4]),
(82, RIT, [p4, p4, p7, p9, s2]),
(83, RIT, [p4, p5, p9, p4, s4]),
(84, MEL, [t3, p6, p6, p8, p3, s4, g1]),
(85, MEL, [p2, p6, p1, p8, s3]),
(86, MEL, [p6, p2, p8, p3, s4]),
(87, MEL, [p8, p3, p1, p2, s2]),
(88, MEL, [t1, g1, t3, g1, g1]),
(89, RIT, [p9, p7, p9, p9, s2]),
(90, RIT, [p9, p7, p7, p9, s3]),
(91, MEL, [t3, g1, p3, p1, p3, p3, s3]),
(92, MEL, [p3, p3, p1, p8, s3]),
(93, MEL, [t2, p2, p1, p3, p8, s3, g1]),
(94, MEL, [t3, g1, p3, p8, p3, p3, s2]),
(95, MEL, [p2, p1, p6, p6, s3]),
(96, RIT, [t1, p5, p9, p4, p9, s3, g2]),
(97, RIT, [t2, p9, p7, p5, p5, s4, g2]),
(98, MEL, [t2, p2, p1, p6, p1, s4, g1]),
(99, RIT, [t3, p9, p5, p9, p9, s1, t3, g2, g2]),
(100, MEL, [t2, g1, g1])}

A.4 Compositions

See Figs. 13.32, 13.33, 13.34, 13.35.
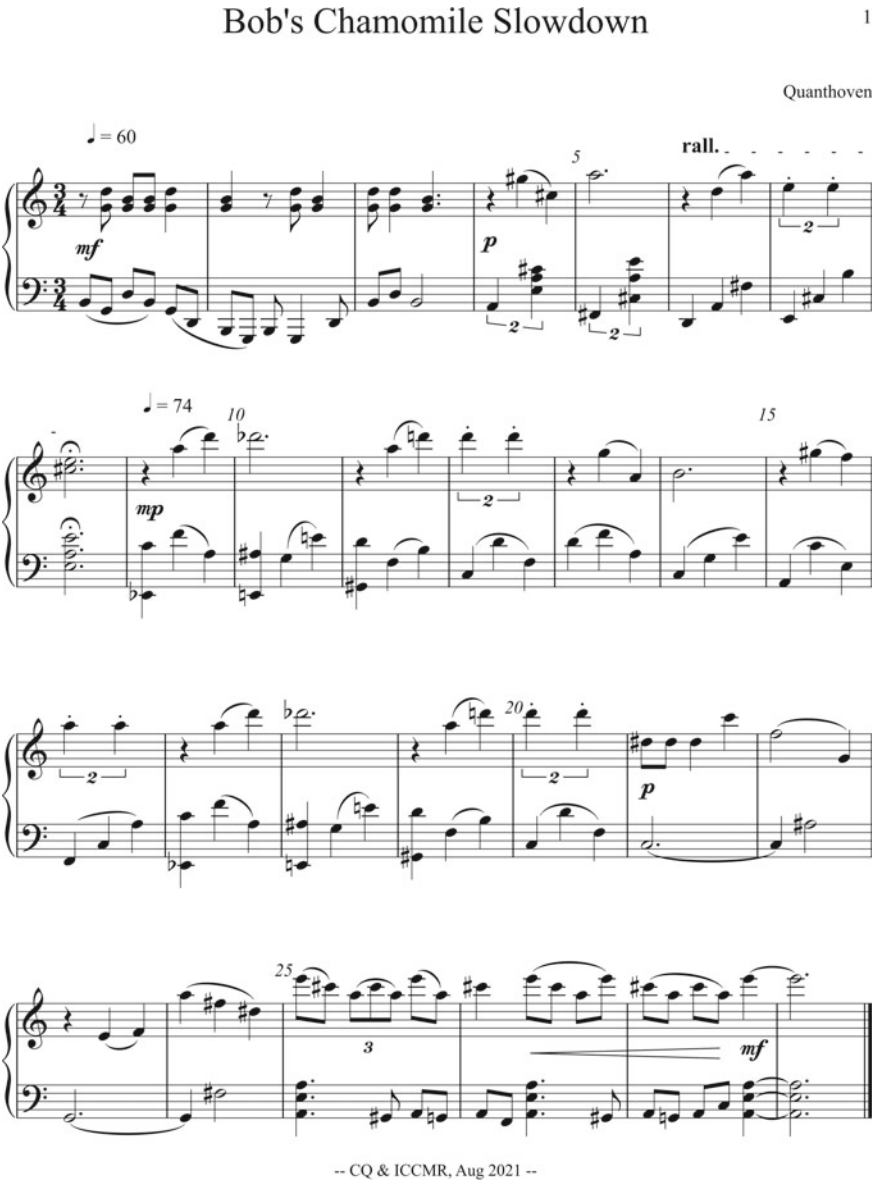


Fig. 13.32 The composition *Bob's Chamomile Slowdown*
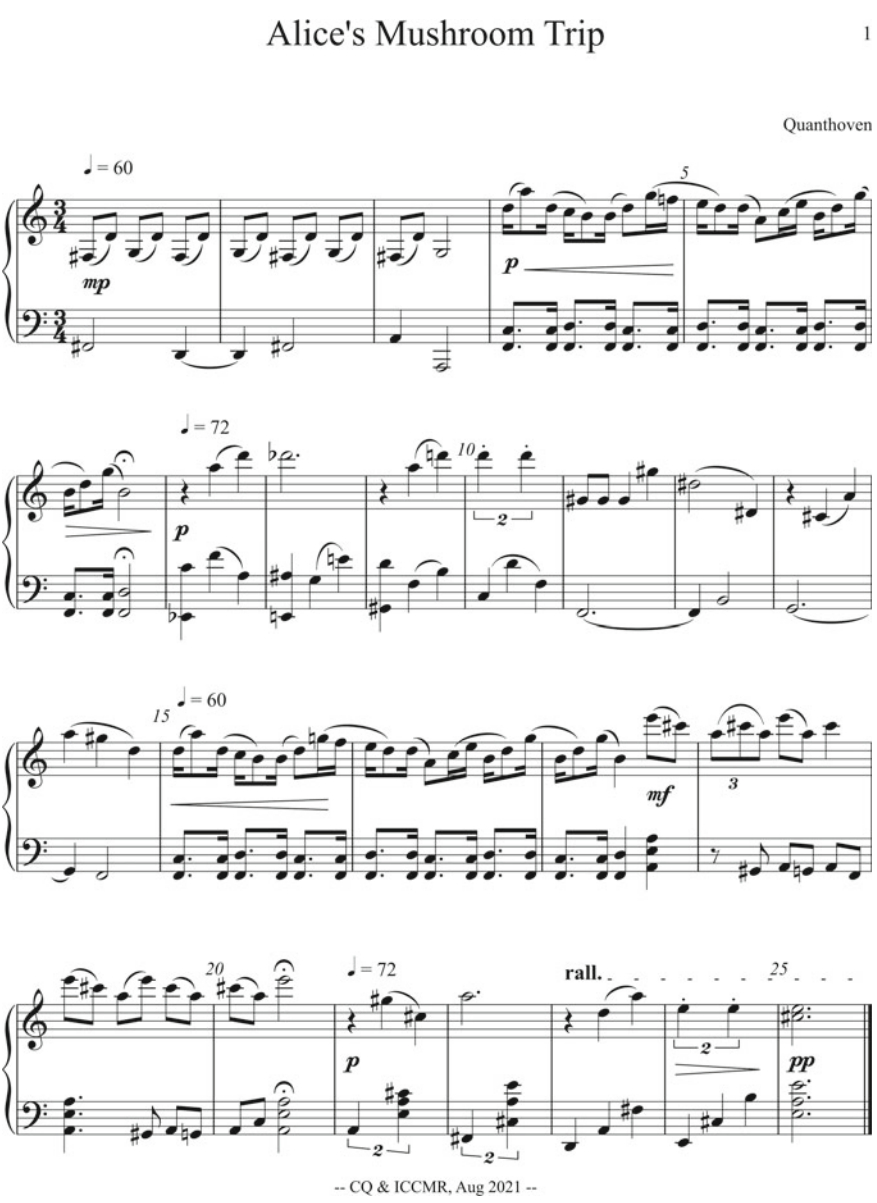


Fig. 13.33 The composition *Alice's Mushroom Trip*
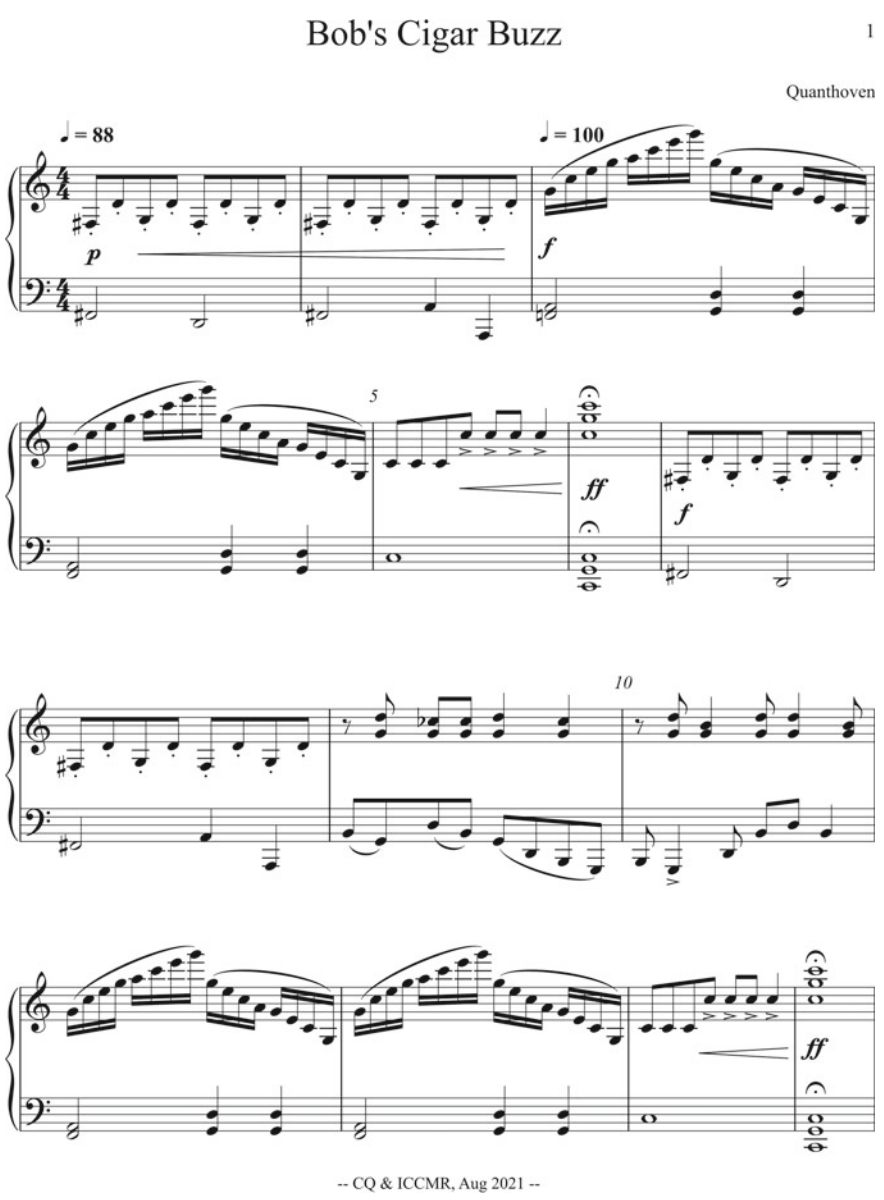


Fig. 13.34 First page of the composition *Bob's Cigar Rush*
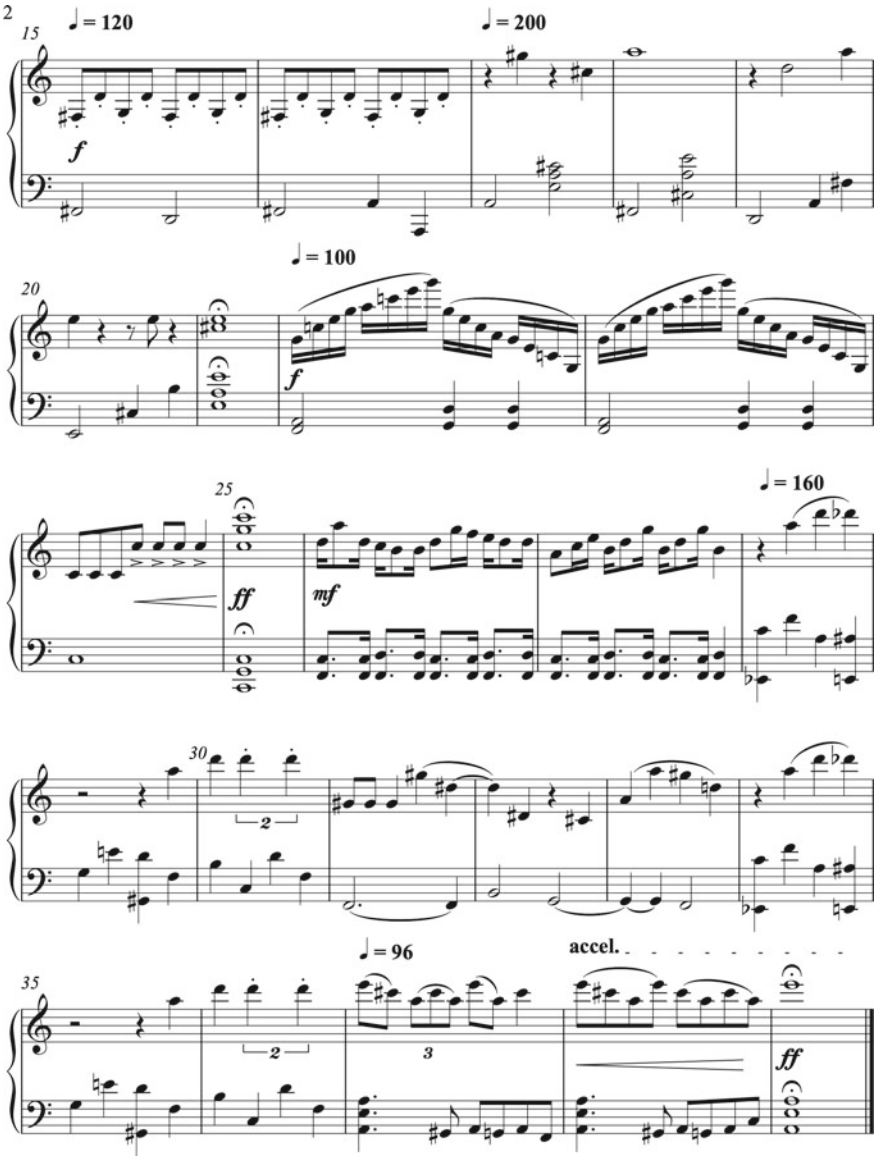


Fig. 13.35 Second page of the composition *Bob's Cigar Rush*

References

1. Abramsky, S., & Coecke, B. (2008). Categorical quantum mechanics. In K. Engesser et al., (Eds.), *Handbook of quantum logic and quantum structures*. Elsevier. arxiv:0808.1023.
2. Adiyansjah, A., Gunawan, A. S., & Suhartono, D. (2019). Music recommender system based on genre using convolutional recurrent neural networks. *Procedia Computer Science, 157*, 99–109. https://doi.org/10.1016/j.procs.2019.08.146
3. Anders, T., & Miranda, E. R. (2009). Interfacing manual and machine composition. *Contemporary Music Review, 28*(2), 133–147. https://doi.org/10.1080/07494460903322422
4. Bakker, D., & Martin, F. H. (2014). Musical chords and emotion: Major and minor triads are processed for emotion. *Cognitive Affective and Behavioral Neuroscience, 15*(1), 15–31. https://doi.org/10.3758/s13415-014-0309-4
5. Baroni, M. (1999). Musical grammar and the study of cognitive processes of composition. *Musicae Scientiae, 3*(1), 3–21.
6. Bod, J. C. (2001). Probabilistic grammars for music. https://www.researchgate.net/publication/2402113_Probabilistic_Grammars_for_Music.
7. de Boer, B. (1999). Evolution and self-organisation in vowel systems. *Evolution of Communication, 3*(1), 79–103.
8. Cherston, J., Hill, E., Goldfarb, S., & Paradiso, J. A. (2016). Musician and mega-machine: Composition driven by real-time particle collision data from the ATLAS detector. In *Proceedings of NIME* (pp. 11–15).
9. Chomsky, N. (1975). *The logical structure of linguistic theory*. Springer.
10. Chomsky, N. (2006). *Language and mind*. Cambridge University Press.
11. Coecke, B., & Duncan, R. (2011). Interacting quantum observables: Categorical algebra and diagrammatics. *New Journal of Physics, 13*(4), 04301. arxiv:0906.4725.
12. Coecke, B., & Kissinger, A. (2017). *Picturing quantum processes: A first course in quantum theory and diagrammatic reasoning*. Cambridge University Press.
13. Coecke, B., Mehrnoosh, S., & Clark, S. (2010). Mathematical foundations for a compositional distributional model of meaning. *Linguistic Analysis, 36*(1–4), 345–384. arxiv:1003.4394.
14. Coecke, B., Grefenstette, E., & Sadrzadeh, M. (2013). Lambek versus Lambek: Functorial vector space semantics and strings diagrams for Lambek calculus. *Annals of Pure and Applied Logic, 164*, 1079–1100. https://doi.org/10.1016/j.apal.2013.05.009.
15. Coecke, B. (2021). Compositionality as we see it, everywhere around us. https://arxiv.org/abs/2110.05327
16. Cope, D. (1991). Recombinant music: Using the computer to explore musical style. *Computer, 24*(7), 22–28. https://doi.org/10.1109/2.84830
17. Cope, D. (2000). *The algorithmic composer*. A-R Editions.
18. Daly, I., Williams, D., Hallowell, J., Hwang, F., Kirke, A., Malik, A., Weaver, J., Miranda, E. R., & Nasuto, S. J. (2015). Music-induced emotions can be predicted from a combination of brain activity and acoustic features. *Brain and Cognition, 101*, 1–11. https://doi.org/10.1016/j.bandc.2015.08.003
19. Dawson, C., Aalto, D., Šimko, J., Vainio, M., & Tervaniemi, M. (2017). Musical sophistication and the effect of complexity on auditory discrimination in finnish speakers. *Frontiers in Neuroscience, 11*, 213. https://doi.org/10.3389/fnins.2017.00213.
20. Edwards, M. (2011). Algorithmic composition: Computational thinking in music. *Communications of the ACM, 54*(7), 58–67.
21. Eliot, L. (1999). *Early intelligence*. Penguin Books.
22. de Felice, G., Tuomi, A., Coecke, B. (2020). Discopy: Monoidal categories in python. In *Applied Category Theory 2020 Conference*. arxiv:2005.02975.
23. Fernandez, J. D., & Vico, F. (2013). Ai methods in algorithmic composition: A comprehensive survey. *Journal of Artificial Intelligence Research, 48*, 513–582.

24. Frank, S. L., Bod, R., & Christiansen, M. H. (2012). How hierarchical is language use? *Proceedings of the Royal Society B, 279*(1747), 4522–4531. https://doi.org/10.1098/rspb.2012.1741
25. Gouyon, P.-Y., Henry, J.-P., & Arnould, J. (2002). *Gene avatars: The neo-Darwinian theory of evolution*. Kluwer Academic/Plenum Publishers.
26. Harley, J. (1995). Generative processes in algorithmic composition: Chaos and music. *Leonardo, 28*(3), 221–224.
27. Hopcroft, J. E., & Ullman, J. D. (1979). *Introduction to automata theory, languages, and computation*. Addison-Wesley.
28. Hsü, K. J., & Hsü, A. J. (1990). Fractal geometry of music. *Proceedings of the National Academy of Sciences, Physics, 87*, 938–941.
29. Jäncke, L. (2012). The relationship between music and language. *Frontiers in Psychology, 3*, 123. https://doi.org/10.3389/fpsyg.2012.001
30. Jordan, R., & Kafalenos, E. (1994). Listening to music: Semiotic and narratological models. *Musikometrik, 6*, 87–115.
31. Kartsaklis, D., Fan, I., Yeung, R., Pearson, A., Lorenz, R., Toumi, A., de Felice, G., Meichanetzidis, K., Clark, S., & Coecke, B. (2021). Lambeq: An efficient high-level python library for quantum NLP.
32. Koelsch, S. (2011). Toward a neural basis of music perception—A review and updated model. *Frontiers in Psychology, 2*, 110. https://doi.org/10.3389/fpsyg.2011.00110
33. Koelsch, S. (2014). Brain correlates of music-evoked emotions. *Nature Reviews Neuroscience, 15*, 170–180. https://doi.org/10.1038/nrn3666
34. Lambek, J. (1999). Type grammar revisited. In *Lecture notes in computer science* (Vol. 582, pp. 1–27). Springer.
35. Lerdhal, F., & Jackendoff, R. (1996). *A generative theory of tonal music*. The MIT Press.
36. Lorenz, R., Pearson, A., Meichanetzidis, K., Kartsaklis, D., & Coecke, B. (2021). QNLP in practice: Running compositional models of meaning on a quantum computer. arxiv:2102.12846.
37. Mann, A. (1965). *The study of counterpoint: From Johann Joseph Fux's Gradus ad Parnassum*. W. W. Norton and Company.
38. Meichanetzidis, K., Toumi, A., de Felice, G., & Coecke, B. (2020). Grammar-aware question-answering on quantum computers. arxiv:2012.03756.
39. Meichanetzidis, K., Gogioso, S., De Felice, G., Chiappori, N., Toumi, A., & Coecke, B. (2021). Quantum natural language processing on near-term quantum computers. *Electronic Proceedings in Theoretical Computer Science, 340*, 213–229. ISSN: 2075-2180. https://doi.org/10.4204/eptcs.340.11.
40. Milovanov, R., & Tervaniemi, M. (2011). The interplay between musical and linguistic aptitudes: A review. *Frontiers in Psychology, 2*, 321. https://doi.org/10.3389/fpsyg.2011.00321
41. Miranda, E. R. (2001). *Composing music with computers*. Focal Press.
42. Miranda, E. R. (2008). Emergent songs by social robots. *Journal of Experimental and Theoretical Artificial Intelligence, 20*(4), 319–334. https://doi.org/10.1080/09528130701664640
43. Miranda, E. R. (2011). *A-life for music: Music and computer models of living systems*. A-R Editions. ISBN: 978-0-89579-673-8.
44. Miranda, E. R. (2014). *Thinking music: The inner workings of a composer's mind*. University of Plymouth Press.
45. Miranda, E. R. (2020). Genetic music system with synthetic biology. *Artificial Life, 26*(3), 1–27. https://doi.org/10.1162/artl_a_00325
46. Miranda, E. R. (Ed.). (2021). *Handbook of artificial intelligence for music foundations, advanced approaches, and developments for creativity*. Springer International Publishing. https://doi.org/10.1007/978-3-030-72116-9.
47. Miranda, E. R., & Biles, J. A. (2007). *Evolutionary computer music*. Springer.
48. Miranda, E. R., Kirby, S., & Todd, P. (2010). On computational models of the evolution of music: From the origins of musical taste to the emergence of grammars. *Contemporary Music Review, 22*(3), 91–111. https://doi.org/10.1080/0749446032000150915

49. Monelle, R. (1992). *Linguistics and semiotics in music*. Harwood Academic Publishers.
50. Moore, J. (2019). The language of moss. https://web.archive.org/web/20090813110752, http://www.thelanguageofmoss.com/. Accessed 02 Apr. 2021.
51. Nielsen, M. A., & Chuang, I. L. (2011). *Quantum computation and quantum information: 10th anniversary edition* (10th ed.). Cambridge University Press. 10.7002176. ISBN: 978-1107002173.
52. Orus, R. (2014). A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics, 349*, 117–158. arxiv.org:1306.2164.
53. Pachet, F., Roy, P., & Carre, B. (2021). Assistive music creation with flow machines: Towards new categories of new. In E. R. Miranda (Ed.), *Handbook of artificial intelligence for music* (pp. 485–520). Springer. ISBN: 978-3-030-72115-2.
54. Patel, A., & Morgan, E. (2016). Exploring cognitive relations between prediction in language and music. *Cognitive Science: A Multidisciplinary Journal, 41*(S2), 303–320. https://doi.org/10.1111/cogs.12411
55. Patel, A. D. (2011). Why would musical training benefit the neural encoding of speech? The opera hypothesis. *Frontiers in Psychology, 2*, 142. https://doi.org/10.3389/fpsyg.2011.00142
56. Perchy, S., & Sarria, G. (2016). Musical composition with stochastic context-free grammars. In *Proceedings of 8th Mexican International Conference on Artificial Intelligence*. https://hal.inria.fr/hal-01257155. Accessed 05 Apr. 2021.
57. Peterson, D. (2015). *The art of language invention*. Penguin Random House.
58. Quanthoven. (2021). *Quantum computer music album*. SoundClick. www.soundclick.com/LudovicoQuanthoven.
59. Rosenkrantz, D. J., & Stearns, R. E. (1970). Properties of deterministic top-down grammars. *Information and Control, 17*(3), 226–256. https://doi.org/10.1016/S0019-9958(70)90446-8. https://www.sciencedirect.com/science/article/pii/S0019995870904468. ISSN: 0019-9958.
60. Sivarajah, S., Dilkes, S., Cowtan, A., Simmons, W., Edgington, A., & Duncan, R. (2020). tlket?: a retargetable compiler for NISQ devices. *Quantum Science and Technology, 6*(1), 014003. https://doi.org/10.1088/2058-9565/ab8e92. ISSN: 2058-9565.
61. Spall, J. C. (1998). Implementation of the simultaneous perturbation algorithm for stochastic optimization. *IEEE Transactions on Aerospace and Electronic Systems, 34*(3), 817–823. https://doi.org/10.1109/7.705889
62. Sutor, R. S. (2019). *Dancing with qubits: How quantum computing works and how it can change the word*. Packt. ISBN: 978-1-838-82736-6.
63. Toumi, A., Yeung, R., & de Felice, G. (2021). Diagrammatic differentiation for quantum machine learning. *Electronic Proceedings in Theoretical Computer Science, 343*, 132–144. https://doi.org/10.4204/eptcs.343.7. ISSN: 2075-2180.
64. Woods, W. A. (1970). Transition networks grammars for natural language analysis. *Communications of the ACM, 13*(10). https://doi.org/10.1145/355598.362773.
65. Yeung, R., & Kartsaklis, D. (2021). A CCG-based version of the DisCoCat framework. https://arxiv.org/abs/2105.07720
66. Yoshikawa, M., Noji, H., & Matsumoto, Y. (2017). A* CCG parsing with a supertag and dependency factored model. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics* (Vol. 1, pp. 277–287). Association for Computational Linguistics. https://doi.org/10.18653/v1/P17-1026. http://aclweb.org/anthology/P17-1026.